

Spring 进阶

JdbcTemplate

JdbcTemplate 是 Spring 提供的一个 JDBC 模板类，是对 JDBC 的封装，简化 JDBC 代码。

当然，你也可以不用，可以让 Spring 集成其它的 ORM 框架，例如：MyBatis、Hibernate 等。

接下来我们简单来学习一下，使用 JdbcTemplate 完成增删改查。

环境准备

- 数据库表：t_user

字段	索引	外键	检查	触发器	选项	注释	SQL 预览							
名称						类型		长度	小数点	不是 null	虚拟	键	注释	
id						int				☒	☐	1	主键自增	
real_name						varchar		255		☐	☐		真实姓名	
age						int				☐	☐		年龄	

- IDEA 中新建模块：spring6-jdbc-demo10
- 引入相关依赖：
 - pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.cqipu.edu</groupId>
  <artifactId>spring6-jdbc-demo10</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <repositories>
    <repository>
      <id>repository.spring.milestone</id>
      <name>Spring Milestone Repository</name>
      <url>https://repo.spring.io/milestone</url>
    </repository>
  </repositories>

  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>6.0.0-M2</version>
    </dependency>
    <dependency>
      <groupId>com.alibaba</groupId>
      <artifactId>druid</artifactId>
      <version>1.2.20</version>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
```

```

        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
    </dependency>
    <!-- 新增的依赖:mysql 驱动-->
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.30</version>
    </dependency>
    <!-- 新增的依赖: spring jdbc, 这个依赖中有JdbcTemplate-->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-jdbc</artifactId>
        <version>6.0.0-M2</version>
    </dependency>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-tx</artifactId>
        <version>6.0.0-M2</version>
    </dependency>
</dependencies>

<properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
</properties>

</project>

```

准备实体类：表t_user对应的实体类User。

- User

```

package org.cqipu.edu.bean;

public class User {
    private Integer id;
    private String realName;
    private Integer age;

    @Override
    public String toString() {
        return "User{" +
            "id=" + id +
            ", realName='" + realName + '\'' +
            ", age=" + age +
            '}';
    }

    public User() {
    }

    public User(Integer id, String realName, Integer age) {
        this.id = id;
        this.realName = realName;
        this.age = age;
    }

    public Integer getId() {
        return id;
    }
}

```

```

public void setId(Integer id) {
    this.id = id;
}

public String getRealName() {
    return realName;
}

public void setRealName(String realName) {
    this.realName = realName;
}

public Integer getAge() {
    return age;
}

public void setAge(Integer age) {
    this.age = age;
}
}

```

编写Spring配置文件：

JdbcTemplate是Spring提供好的类，这类的完整类名是：`org.springframework.jdbc.core.JdbcTemplate`

我们怎么使用这个类呢？new对象就可以了。怎么new对象，Spring最在行了。直接将这个类配置到Spring配置文件中，纳入Bean管理即可。

- spring.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate"></bean>
</beans>

```

我们来看一下这个JdbcTemplate源码：

- JdbcTemplate.java

```

public class JdbcTemplate extends JdbcAccessor implements JdbcOperations {
    private static final String RETURN_RESULT_SET_PREFIX = "#result-set-";
    private static final String RETURN_UPDATE_COUNT_PREFIX = "#update-count-";
    private boolean ignoreWarnings = true;
    private int fetchSize = -1;
    private int maxRows = -1;
    private int queryTimeout = -1;
    private boolean skipResultsProcessing = false;
    private boolean skipUndeclaredResults = false;
    private boolean resultsMapCaseInsensitive = false;

    public JdbcTemplate() {
    }

    public JdbcTemplate(DataSource dataSource) {
        this.setDataSource(dataSource);
        this.afterPropertiesSet();
    }
}

```

- JdbcAccessor.java

```
public abstract class JdbcAccessor implements InitializingBean {
    private static final boolean shouldIgnoreXml = SpringProperties.getFlag("spring.xml.ignore");
    protected final Log logger = LoggerFactory.getLog(this.getClass());
    @Nullable
    private DataSource dataSource;
    @Nullable
    private volatile SQLExceptionTranslator exceptionTranslator;
    private boolean lazyInit = true;

    public JdbcAccessor() {
    }
}
```

可以看到JdbcTemplate中有一个DataSource属性，这个属性是数据源，我们都知道连接数据库需要Connection对象，而生成Connection对象是数据源负责的。所以我们需要给JdbcTemplate设置数据源属性。

所有的数据源都是要实现javax.sql.DataSource接口的。这个数据源可以自己写一个，也可以用写好的，比如：阿里巴巴的德鲁伊连接池，c3p0，dbcp等。我们这里自己先手写一个数据源。

- 自己写的数据源

```
package org.cqipu.edu.jdbc;
import javax.sql.DataSource;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.SQLFeatureNotSupportedException;
import java.util.logging.Logger;

public class MyDataSource implements DataSource {
    // 添加4个属性
    private String driver;
    private String url;
    private String username;
    private String password;

    // 提供4个setter方法
    public void setDriver(String driver) {
        this.driver = driver;
    }

    public void setUrl(String url) {
        this.url = url;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    // 重点写怎么获取Connection对象就行。其他方法不用管。
    @Override
    public Connection getConnection() throws SQLException {
        try {
            Class.forName(driver);
            Connection conn = DriverManager.getConnection(url, username, password);
        }
    }
}
```



```

        return conn;
    } catch (Exception e) {
        e.printStackTrace();
    }
    return null;
}

@Override
public Connection getConnection(String username, String password) throws SQLException {
    return null;
}

@Override
public PrintWriter getLogWriter() throws SQLException {
    return null;
}

@Override
public void setLogWriter(PrintWriter out) throws SQLException {

}

@Override
public void setLoginTimeout(int seconds) throws SQLException {

}

@Override
public int getLoginTimeout() throws SQLException {
    return 0;
}

@Override
public Logger getParentLogger() throws SQLFeatureNotSupportedException {
    return null;
}

@Override
public <T> T unwrap(Class<T> iface) throws SQLException {
    return null;
}

@Override
public boolean isWrapperFor(Class<?> iface) throws SQLException {
    return false;
}
}

```

写完数据源，我们需要把这个数据源传递给JdbcTemplate。因为JdbcTemplate中有一个DataSource属性：

- spring.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="myDataSource" class="org.cqipu.edu.jdbc.MyDataSource">
        <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
        <property name="url" value="jdbc:mysql://localhost:3306/spring6"/>
        <property name="username" value="root"/>
        <property name="password" value="20060329"/>
    </bean>

```

```
<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="myDataSource"/>
</bean>
</beans>
```

新增

编写测试程序：

- 测试insert

```
package org.cqipu.edu.test;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.jdbc.core.JdbcTemplate;

public class JdbcTest {
    @Test
    public void testInsert(){
        // 获取JdbcTemplate对象
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
        JdbcTemplate jdbcTemplate = applicationContext.getBean("jdbcTemplate", JdbcTemplate.class);
        // 执行插入操作
        // 注意: insert delete update的sql语句, 都是执行update方法。
        String sql = "insert into t_user(id,real_name,age) values(?,?,?)";
        int count = jdbcTemplate.update(sql, null, "张三", 30);
        System.out.println("插入的记录条数: " + count);
    }
}
```

update方法有两个参数：

- 第一个参数：要执行的SQL语句。（SQL语句中可能会有占位符？）
- 第二个参数：可变长参数，参数的个数可以是0个，也可以是多个。一般是SQL语句中有几个问号，则对应几个参数。

✓ 测试 已通过: 1共 1 个测试 - 941毫秒

```
D:\Java21\bin\java.exe ...
插入的记录条数: 1
```

修改

- 测试程序

```
@Test
public void testUpdate(){
    // 获取JdbcTemplate对象
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
    JdbcTemplate jdbcTemplate = applicationContext.getBean("jdbcTemplate", JdbcTemplate.class);
    // 执行更新操作
    String sql = "update t_user set real_name = ?, age = ? where id = ?";
    int count = jdbcTemplate.update(sql, "张三丰", 55, 1);
    System.out.println("更新的记录条数: " + count);
}
```

✓ 测试 已通过: 1共 1 个测试 - 585毫秒

D:\Java21\bin\java.exe ...

更新的记录条数: 1

删除

- 测试程序

● ● ●

```
@Test
public void testDelete(){
    // 获取JdbcTemplate对象
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
    JdbcTemplate jdbcTemplate = applicationContext.getBean("jdbcTemplate", JdbcTemplate.class);
    // 执行delete
    String sql = "delete from t_user where id = ?";
    int count = jdbcTemplate.update(sql, 1);
    System.out.println("删除了几条记录: " + count);
}
```

✓ 测试 已通过: 1共 1 个测试 - 579毫秒

D:\Java21\bin\java.exe ...

删除了几条记录: 1

查询一个对象

● ● ●

```
@Test
public void testSelectOne(){
    // 获取JdbcTemplate对象
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
    JdbcTemplate jdbcTemplate = applicationContext.getBean("jdbcTemplate", JdbcTemplate.class);
    // 执行select
    String sql = "select id, real_name, age from t_user where id = ?";
    User user = jdbcTemplate.queryForObject(sql, new BeanPropertyRowMapper<>(User.class), 2);
    System.out.println(user);
}
```

✓ 测试 已通过: 1共 1 个测试 - 687毫秒

D:\Java21\bin\java.exe ...

User{id=2, realName='张三', age=20}

queryForObject方法三个参数:

- 第一个参数: sql语句
- 第二个参数: Bean属性值和数据库记录行的映射对象。在构造方法中指定映射的对象类型。
- 第三个参数: 可变长参数, 给sql语句的占位符问号传值。

查询多个对象



@Test

```
public void testSelectAll(){
    // 获取JdbcTemplate对象
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
    JdbcTemplate jdbcTemplate = applicationContext.getBean("jdbcTemplate", JdbcTemplate.class);
    // 执行select
    String sql = "select id, real_name, age from t_user";
    List<User> users = jdbcTemplate.query(sql, new BeanPropertyRowMapper<>(User.class));
    System.out.println(users);
}
```

✓ 测试 已通过: 1共 1 个测试 - 755毫秒

D:\Java21\bin\java.exe ...

[User{id=2, realName='张三', age=20}, User{id=3, realName='李四', age=22}, User{id=4, realName='王五', age=25}, User{id=5, realName='赵六', age=19}, User{id=6, realName='孙七', age=28}]

查询一个值



@Test

```
public void testSelectOneValue(){
    // 获取JdbcTemplate对象
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
    JdbcTemplate jdbcTemplate = applicationContext.getBean("jdbcTemplate", JdbcTemplate.class);
    // 执行select
    String sql = "select count(1) from t_user";
    Integer count = jdbcTemplate.queryForObject(sql, int.class); // 这里用Integer.class也可以
    System.out.println("总记录条数: " + count);
}
```

✓ 测试 已通过: 1共 1 个测试 - 573毫秒

D:\Java21\bin\java.exe ...

总记录条数: 5

批量添加



@Test

```
public void testAddBatch(){
    // 获取JdbcTemplate对象
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
    JdbcTemplate jdbcTemplate = applicationContext.getBean("jdbcTemplate", JdbcTemplate.class);
    // 批量添加
    String sql = "insert into t_user(id,real_name,age) values(?,?,?)";

    Object[] objs1 = {null, "小花", 20};
    Object[] objs2 = {null, "小明", 21};
    Object[] objs3 = {null, "小刚", 22};
    List<Object[]> list = new ArrayList<>();
    list.add(objs1);
    list.add(objs2);
    list.add(objs3);

    int[] count = jdbcTemplate.batchUpdate(sql, list);
}
```

```
System.out.println(Arrays.toString(count));  
}
```

✓ 测试 已通过: 1共 1 个测试 - 587毫秒

```
D:\Java21\bin\java.exe ...  
[1, 1, 1]
```

批量修改

```
@Test  
public void testUpdateBatch(){  
    // 获取JdbcTemplate对象  
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");  
    JdbcTemplate jdbcTemplate = applicationContext.getBean("jdbcTemplate", JdbcTemplate.class);  
    // 批量修改  
    String sql = "update t_user set real_name = ?, age = ? where id = ?";  
    Object[] objs1 = {"小花11", 10, 2};  
    Object[] objs2 = {"小明22", 12, 3};  
    Object[] objs3 = {"小刚33", 9, 4};  
    List<Object[]> list = new ArrayList<>();  
    list.add(objs1);  
    list.add(objs2);  
    list.add(objs3);  
  
    int[] count = jdbcTemplate.batchUpdate(sql, list);  
    System.out.println(Arrays.toString(count));  
}
```

✓ 测试 已通过: 1共 1 个测试 - 566毫秒

```
D:\Java21\bin\java.exe ...  
[1, 1, 1]
```

批量删除

```
@Test  
public void testDeleteBatch(){  
    // 获取JdbcTemplate对象  
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");  
    JdbcTemplate jdbcTemplate = applicationContext.getBean("jdbcTemplate", JdbcTemplate.class);  
    // 批量删除  
    String sql = "delete from t_user where id = ?";  
    Object[] objs1 = {2};  
    Object[] objs2 = {3};  
    Object[] objs3 = {4};  
    List<Object[]> list = new ArrayList<>();  
    list.add(objs1);  
    list.add(objs2);  
    list.add(objs3);  
    int[] count = jdbcTemplate.batchUpdate(sql, list);  
    System.out.println(Arrays.toString(count));  
}
```

✓ 测试 已通过: 1共 1 个测试 - 555毫秒

```
D:\Java21\bin\java.exe ...  
[1, 1, 1]
```

使用回调函数

```
package org.cqipu.edu.test;
import org.junit.Test;
import org.cqipu.edu.bean.User;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.dao.DataAccessException;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.PreparedStatementCallback;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class JdbcTest {
    @Test
    public void testCallback(){
        // 获取JdbcTemplate对象
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
        JdbcTemplate jdbcTemplate = applicationContext.getBean("jdbcTemplate", JdbcTemplate.class);
        String sql = "select id, real_name, age from t_user where id = ?";

        User user = jdbcTemplate.execute(sql, new PreparedStatementCallback<User>() {
            @Override
            public User doInPreparedStatement(PreparedStatement ps) throws SQLException, DataAccessException {
                User user = null;
                ps.setInt(1, 5);
                ResultSet rs = ps.executeQuery();
                if (rs.next()) {
                    user = new User();
                    user.setId(rs.getInt("id"));
                    user.setRealName(rs.getString("real_name"));
                    user.setAge(rs.getInt("age"));
                }
                return user;
            }
        });
        System.out.println(user);
    }
}
```

✓ 测试 已通过: 1共 1 个测试 - 560毫秒

```
D:\Java21\bin\java.exe ...
User{id=5, realName='赵六', age=19}
```

使用德鲁伊连接池

之前数据源是用我们自己写的。也可以使用别人写好的。例如比较牛的德鲁伊连接池。

第一步：引入德鲁伊连接池的依赖。（毕竟是别人写的）

- 德鲁伊依赖 pom.xml

```
<dependency>
  <groupId>com.alibaba</groupId>
  <artifactId>druid</artifactId>
  <version>1.1.8</version>
</dependency>
```

第二步：将德鲁伊中的数据源配置到spring配置文件中。和配置我们自己写的一样。

- spring2.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

  <bean id="druidDataSource" class="com.alibaba.druid.pool.DruidDataSource">
    <property name="driverClassName" value="com.mysql.cj.jdbc.Driver"/>
    <property name="url" value="jdbc:mysql://localhost:3306/spring6"/>
    <property name="username" value="root"/>
    <property name="password" value="20060329"/>
  </bean>

  <bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="druidDataSource"/>
  </bean>
</beans>
```

测试程序

```
@Test
public void testCallback2(){
    // 获取JdbcTemplate对象
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring2.xml");
    JdbcTemplate jdbcTemplate = applicationContext.getBean("jdbcTemplate", JdbcTemplate.class);
    String sql = "select id, real_name, age from t_user where id = ?";

    User user = jdbcTemplate.execute(sql, new PreparedStatementCallback<User>() {
        @Override
        public User doInPreparedStatement(PreparedStatement ps) throws SQLException, DataAccessException {
            User user = null;
            ps.setInt(1, 5);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                user = new User();
                user.setId(rs.getInt("id"));
                user.setRealName(rs.getString("real_name"));
                user.setAge(rs.getInt("age"));
            }
            return user;
        }
    });
    System.out.println(user);
}
```

✓ 测试 已通过: 1共 1 个测试 - 782毫秒

```
D:\Java21\bin\java.exe ...
User{id=5, realName='赵六', age=19}
```

GoF之代理模式

对代理模式的理解

生活场景1：牛村的牛二看上了隔壁村小花，牛二不好意思直接找小花，于是牛二找来了媒婆王妈妈。这里面就有一个非常典型的代理模式。牛二不能和小花直接对接，只能找一个中间人。其中王妈妈是代理类，牛二是目标类。王妈妈代替牛二和小花先见个面。（现实生活中的婚介所）【在程序中，对象A和对象B无法直接交互时。】

生活场景2：你刚到北京，要租房子，可以自己找，也可以找链家帮你找。其中链家是代理类，你是目标类。你们两个都有共同的行为：找房子。不过链家除了满足你找房子，另外会收取一些费用的。（现实生活中的房产中介）【在程序中，功能需要增强时。】

西游记场景：八戒和高小姐的故事。八戒要强抢民女高翠兰。悟空得知此事之后怎么做的？悟空幻化成高小姐的模样。代替高小姐与八戒会面。其中八戒是客户端程序。悟空是代理类。高小姐是目标类。那天夜里，在八戒眼里，眼前的就是高小姐，对于八戒来说，他是不知道眼前的高小姐是悟空幻化的，在他内心里这就是高小姐。所以悟空代替高小姐和八戒亲了嘴儿。这是非常典型的代理模式实现的保护机制。代理模式中有一个非常重要的特点：对于客户端程序来说，使用代理对象时就像在使用目标对象一样。【在程序中，目标需要被保护时】

业务场景：系统中有A、B、C三个模块，使用这些模块的前提是需要用户登录，也就是说在A模块中要编写判断登录的代码，B模块中也要编写，C模块中还要编写，这些判断登录的代码反复出现，显然代码没有得到复用，可以为A、B、C三个模块提供一个代理，在代理当中写一次登录判断即可。代理的逻辑是：请求来了之后，判断用户是否登录了，如果已经登录了，则执行对应的目标，如果没有登录则跳转到登录页面。【在程序中，目标不但受到保护，并且代码也得到了复用。】

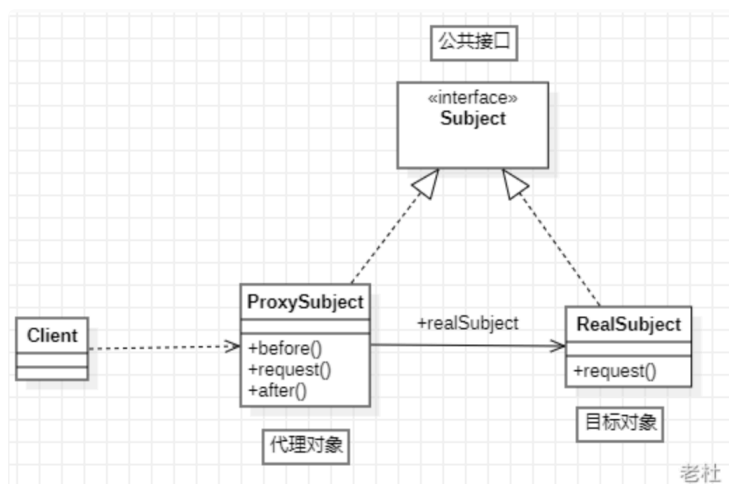
代理模式是GoF23种设计模式之一。属于结构型设计模式。

代理模式的作用是：为其他对象提供一种代理以控制对这个对象的访问。在某些情况下，一个客户不想或者不能直接引用一个对象，此时可以通过一个称之为“代理”的第三者来实现间接引用。代理对象可以在客户端和目标对象之间起到中介的作用，并且可以通过代理对象去掉客户不应该看到的内容和服务或者添加客户需要的额外服务。通过引入一个新的对象来实现对真实对象的操作或者将新的对象作为真实对象的一个替身，这种实现机制即为代理模式，通过引入代理对象来间接访问一个对象，这就是代理模式的模式动机。

代理模式中的角色：

- 代理类（代理主题）
- 目标类（真实主题）
- 代理类和目标类的公共接口（抽象主题）：客户端在使用代理类时就像在使用目标类，不被客户端所察觉，所以代理类和目标类要有共同的行为，也就是实现共同的接口。

代理模式的类图：



代理模式在代码实现上，包括两种形式：

- 静态代理
- 动态代理

静态代理

现在有这么一个接口和实现类：

- OrderService接口

```
package org.cqipu.edu.service;

public interface OrderService {
    /**
     * 生成订单
     */
    void generate();

    /**
     * 查看订单详情
     */
    void detail();

    /**
     * 修改订单
     */
    void modify();
}
```

- OrderService接口的实现类

```
package org.cqipu.edu.service.impl;
import org.cqipu.edu.service.OrderService;

public class OrderServiceImpl implements OrderService {
    @Override
    public void generate() {
        try {
            Thread.sleep(1234);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("订单已生成");
    }

    @Override
    public void detail() {
        try {
            Thread.sleep(2541);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("订单信息如下: *****");
    }

    @Override
    public void modify() {
        try {
            Thread.sleep(1010);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

```

    }
    System.out.println("订单已修改");
}
}

```

其中Thread.sleep()方法的调用是为了模拟操作耗时。

项目已上线，并且运行正常，只是客户反馈系统有一些地方运行较慢，要求项目组对系统进行优化。于是项目负责人就下达了这个需求。首先需要搞清楚是哪些业务方法耗时较长，于是让我们统计每个业务方法所耗费的时长。如果是你，你该怎么做呢？

- 第一种方案：直接修改Java源代码，在每个业务方法中添加统计逻辑，如下：

```

package org.cqipu.edu.service.impl;
import org.cqipu.edu.service.OrderService;

public class OrderServiceImpl implements OrderService {
    @Override
    public void generate() {
        Long begin = System.currentTimeMillis();
        try {
            Thread.sleep(1234);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("订单已生成");
        Long end = System.currentTimeMillis();
        System.out.println("耗时时长" + (end - begin) + "毫秒");
    }

    @Override
    public void detail() {
        Long begin = System.currentTimeMillis();
        try {
            Thread.sleep(2541);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("订单信息如下: *****");
        Long end = System.currentTimeMillis();
        System.out.println("耗时时长" + (end - begin) + "毫秒");
    }

    @Override
    public void modify() {
        Long begin = System.currentTimeMillis();
        try {
            Thread.sleep(1010);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("订单已修改");
        Long end = System.currentTimeMillis();
        System.out.println("耗时时长" + (end - begin) + "毫秒");
    }
}

```

需求可以满足，但显然是违背了OCP开闭原则。这种方案不可取。

- 第二种方案：编写一个子类继承OrderServiceImpl，在子类中重写每个方法，代码如下：
 - OrderServiceImpl的子类

```

package org.cqipu.edu.service.impl;

public class OrderServiceImplSub extends OrderServiceImpl{
    @Override
    public void generate() {
        Long begin = System.currentTimeMillis();
        super.generate();
        Long end = System.currentTimeMillis();
        System.out.println("耗时" + (end - begin) + "毫秒");
    }

    @Override
    public void detail() {
        Long begin = System.currentTimeMillis();
        super.detail();
        Long end = System.currentTimeMillis();
        System.out.println("耗时" + (end - begin) + "毫秒");
    }

    @Override
    public void modify() {
        Long begin = System.currentTimeMillis();
        super.modify();
        Long end = System.currentTimeMillis();
        System.out.println("耗时" + (end - begin) + "毫秒");
    }
}

```

这种方式可以解决，但是存在两个问题：

第一个问题：假设系统中有100个这样的业务类，需要提供100个子类，并且之前写好的创建Service对象的代码，都要修改为创建子类对象。

第二个问题：由于采用了继承的方式，导致代码之间的耦合度较高。

这种方案也不可取。

- 第三种方案：使用代理模式（这里采用静态代理）
 - 可以为OrderService接口提供一个代理类。

```

package org.cqipu.edu.service;

public class OrderServiceProxy implements OrderService{ // 代理对象

    // 目标对象
    private OrderService orderService;

    // 通过构造方法将目标对象传递给代理对象
    public OrderServiceProxy(OrderService orderService) {
        this.orderService = orderService;
    }

    @Override
    public void generate() {
        Long begin = System.currentTimeMillis();
        // 执行目标对象的目标方法
        orderService.generate();
        Long end = System.currentTimeMillis();
        System.out.println("耗时" + (end - begin) + "毫秒");
    }

    @Override
    public void detail() {

```

```

        Long begin = System.currentTimeMillis();
        // 执行目标对象的目标方法
        orderService.detail();
        Long end = System.currentTimeMillis();
        System.out.println("耗时" + (end - begin) + "毫秒");
    }

    @Override
    public void modify() {
        Long begin = System.currentTimeMillis();
        // 执行目标对象的目标方法
        orderService.modify();
        Long end = System.currentTimeMillis();
        System.out.println("耗时" + (end - begin) + "毫秒");
    }
}

```

这种方式的优点：符合OCP开闭原则，同时采用的是关联关系，所以程序的耦合度较低。所以这种方案是被推荐的。

- 编写客户端程序：

```

package org.cqipu.edu.test;
import org.cqipu.edu.service.OrderService;
import org.cqipu.edu.service.OrderServiceProxy;
import org.cqipu.edu.service.impl.OrderServiceImpl;

public class Client {
    public static void main(String[] args) {
        // 创建目标对象
        OrderService target = new OrderServiceImpl();
        // 创建代理对象
        OrderService proxy = new OrderServiceProxy(target);
        // 调用代理对象的代理方法
        proxy.generate();
        proxy.modify();
        proxy.detail();
    }
}

```

执行结果

```

D:\Java21\bin\java.exe ...
订单已生成
耗费时长1235毫秒
耗时1239毫秒
订单已修改
耗费时长1020毫秒
耗时1020毫秒
订单信息如下：*****
耗费时长2542毫秒
耗时2542毫秒

```

以上就是代理模式中的静态代理，其中OrderService接口是代理类和目标类的共同接口。OrderServiceImpl是目标类。OrderServiceProxy是代理类。

大家思考一下：如果系统中业务接口很多，一个接口对应一个代理类，显然也是不合理的，会导致类爆炸。怎么解决这个问题？动态代理可以解决。因为在动态代理中可以在内存中动态的为我们生成代理类的字节码。代理类不需要我们写了。类爆炸解决了，而且代码只需要写一次，代码也会得到复用。

动态代理

在程序运行阶段，在内存中动态生成代理类，被称为动态代理，目的是为了减少代理类的数量。解决代码复用的问题。在内存当中动态生成类的技术常见的包括：

- JDK动态代理技术：只能代理接口。
- CGLIB动态代理技术：CGLIB(Code Generation Library)是一个开源项目。是一个强大的，高性能，高质量的Code生成类库，它可以在运行期扩展Java类与实现Java接口。它既可以代理接口，又可以代理类，底层是通过继承的方式实现的。性能比JDK动态代理要好。（底层有一个小而快的字节码处理框架ASM。）
- Javassist动态代理技术：Javassist是一个开源的分析、编辑和创建Java字节码的类库。是由东京工业大学的数学和计算机科学系的Shigeru Chiba（千叶 滋）所创建的。它已加入了开放源代码JBoss 应用服务器项目，通过使用Javassist对字节码操作为JBoss实现动态"AOP"框架。

JDK动态代理

我们还是使用静态代理中的例子：一个接口和一个实现类。

- OrderService接口

```
package org.cqipu.edu.service;

public interface OrderService {

    /**
     * 生成订单
     */
    void generate();

    /**
     * 查看订单详情
     */
    void detail();

    /**
     * 修改订单
     */
    void modify();
}
```

- OrderService接口实现类

```
package org.cqipu.edu.service.impl;

import org.cqipu.edu.service.OrderService;

public class OrderServiceImpl implements OrderService {

    @Override
    public void generate() {
        try {
            Thread.sleep(1234);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("订单已生成");
    }

    @Override
    public void detail() {
        try {
            Thread.sleep(2541);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

```

    }
    System.out.println("订单信息如下: *****");
}

@Override
public void modify() {
    try {
        Thread.sleep(1010);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    System.out.println("订单已修改");
}
}

```

我们在静态代理的时候，除了以上一个接口和一个实现类之外，是不是要写一个代理类UserServiceProxy呀！在动态代理中UserServiceProxy代理类是可以动态生成的。这个类不需要写。我们直接写客户端程序即可：

- 客户端程序

```

package org.cqipu.edu.test;
import org.cqipu.edu.service.OrderService;
import org.cqipu.edu.service.impl.OrderServiceImpl;
import java.lang.reflect.Proxy;

public class Client {
    public static void main(String[] args) {
        // 第一步：创建目标对象
        OrderService target = new OrderServiceImpl();
        // 第二步：创建代理对象
        OrderService orderServiceProxy = Proxy.newProxyInstance(target.getClass().getClassLoader(),
target.getClass().getInterfaces(), 调用处理器对象);
        // 第三步：调用代理对象的代理方法
        orderServiceProxy.detail();
        orderServiceProxy.modify();
        orderServiceProxy.generate();
    }
}

```

以上第二步创建代理对象是需要大家理解的：

```

OrderService orderServiceProxy = Proxy.newProxyInstance(target.getClass().getClassLoader(),
target.getClass().getInterfaces(), 调用处理器对象);

```

这行代码做了两件事：

- 第一件事：在内存中生成了代理类的字节码
- 第二件事：创建代理对象

Proxy类全名：java.lang.reflect.Proxy。这是JDK提供的一个类（所以称为JDK动态代理）。主要是通过这个类在内存中生成代理类的字节码。

其中newProxyInstance()方法有三个参数：

- 第一个参数：类加载器。在内存中生成了字节码，要想执行这个字节码，也是需要先把这个字节码加载到内存当中的。所以要指定使用哪个类加载器加载。
- 第二个参数：接口类型。代理类和目标类实现相同的接口，所以要通过这个参数告诉JDK动态代理生成的类要实现哪些接口。
- 第三个参数：调用处理器。这是一个JDK动态代理规定的接口，接口全名：java.lang.reflect.InvocationHandler。显然这是一个回调接口，也就是说调用这个接口中方法的程序已经写好了，就差这个接口的实现类了。

所以接下来我们要写一下java.lang.reflect.InvocationHandler接口的实现类，并且实现接口中的方法，代码如下：

- TimerInvocationHandler

```
package org.cqipu.edu.service;
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;

public class TimerInvocationHandler implements InvocationHandler {
    @Override
    public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
        return null;
    }
}
```

InvocationHandler接口中有一个方法invoke，这个invoke方法上有三个参数：

- 第一个参数：Object proxy。代理对象。设计这个参数只是为了后期的方便，如果想在invoke方法中使用代理对象的话，尽管通过这个参数来使用。
- 第二个参数：Method method。目标方法。
- 第三个参数：Object[] args。目标方法调用时要传的参数。

我们将来肯定是要调用“目标方法”的，但要调用目标方法的话，需要“目标对象”的存在，“目标对象”从哪儿来呢？我们可以给TimerInvocationHandler提供一个构造方法，可以通过这个构造方法传过来“目标对象”，代码如下：

```
package org.cqipu.edu.service;
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;
public class TimerInvocationHandler implements InvocationHandler {
    // 目标对象
    private Object target;

    // 通过构造方法来传目标对象
    public TimerInvocationHandler(Object target) {
        this.target = target;
    }

    @Override
    public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
        return null;
    }
}
```

有了目标对象我们就可以在invoke()方法中调用目标方法了。代码如下：

- TimerInvocationHandler

```
package org.cqipu.edu.service;
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;
public class TimerInvocationHandler implements InvocationHandler {
    // 目标对象
    private Object target;

    // 通过构造方法来传目标对象
    public TimerInvocationHandler(Object target) {
        this.target = target;
    }
}
```

```

@Override
public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
    // 目标执行之前增强。
    long begin = System.currentTimeMillis();
    // 调用目标对象的目标方法
    Object retValue = method.invoke(target, args);
    // 目标执行之后增强。
    long end = System.currentTimeMillis();
    System.out.println("耗时" + (end - begin) + "毫秒");
    // 一定要记得返回哦。
    return retValue;
}
}

```

到此为止，调用处理器就完成了。接下来，应该继续完善Client程序：

- 客户端程序

```

package org.cqipu.edu.test;
import org.cqipu.edu.service.OrderService;
import org.cqipu.edu.service.impl.OrderServiceImpl;
import java.lang.reflect.Proxy;
import org.cqipu.edu.service.TimerInvocationHandler;
public class Client {
    public static void main(String[] args) {
        // 创建目标对象
        OrderService target = new OrderServiceImpl();
        // 创建代理对象
        OrderService orderServiceProxy = (OrderService) Proxy.newProxyInstance(target.getClass().getClassLoader(),
            target.getClass().getInterfaces(),
            new TimerInvocationHandler(target));
        // 调用代理对象的代理方法
        orderServiceProxy.detail();
        orderServiceProxy.modify();
        orderServiceProxy.generate();
    }
}

```

大家可能会比较好奇：那个InvocationHandler接口中的invoke()方法没看见在哪里调用呀？

注意：当你调用代理对象的代理方法的时候，注册在InvocationHandler接口中的invoke()方法会被调用。也就是上面代码第24 25 26行，这三行代码中任意一行代码执行，注册在InvocationHandler接口中的invoke()方法都会被调用。

- 执行结果：

```

D:\Java21\bin\java.exe ...
订单信息如下: *****
耗时2544毫秒
订单已修改
耗时1025毫秒
订单已生成
耗时1236毫秒

进程已结束，退出代码为 0

```

学到这里可能会感觉有点懵，折腾半天，到最后这不是还得写一个接口的实现类吗？没省劲儿呀？

你要这样想就错了!!!!

我们可以看到，不管你有多少个Service接口，多少个业务类，这个TimerInvocationHandler接口是不是只需要写一次就行了，代码是不是得到复用了!!!!

而且最重要的是，以后程序员只需要关注核心业务的编写了，像这种统计时间的代码根本不需要关注。因为这种统计时间的代码只需要在调用处理器中编写一次即可。

到这里，JDK动态代理的原理就结束了。

不过我们看以下这个代码确实有点繁琐，对于客户端来说，用起来不方便：

```
public class Client {  
    public static void main(String[] args) {  
        // 创建目标对象  
        OrderService target = new OrderServiceImpl();  
        // 创建代理对象  
        OrderService orderServiceProxy = (OrderService) Proxy.newProxyInstance(target.getClass().getClassLoader(),  
                                                                                 target.getClass().getInterfaces(),  
                                                                                 new TimerInvocationHandler(target));  
        // 调用代理对象的代理方法  
        orderServiceProxy.detail();  
    }  
}
```

老

我们可以提供一个工具类：ProxyUtil，封装一个方法：

- 工具类

```
package org.cqipu.edu.util;  
import org.cqipu.edu.service.TimerInvocationHandler;  
  
import java.lang.reflect.Proxy;  
  
public class ProxyUtil {  
    public static Object newProxyInstance(Object target) {  
        return Proxy.newProxyInstance(target.getClass().getClassLoader(),  
                                       target.getClass().getInterfaces(),  
                                       new TimerInvocationHandler(target));  
    }  
}
```

这样客户端代码就不需要写那么繁琐了：

- 客户端程序

```
package org.cqipu.edu.test;  
import org.cqipu.edu.service.OrderService;  
import org.cqipu.edu.service.impl.OrderServiceImpl;  
import org.cqipu.edu.util.ProxyUtil;  
  
public class Client {  
    public static void main(String[] args) {  
        // 创建目标对象  
        OrderService target = new OrderServiceImpl();  
        // 创建代理对象  
        OrderService orderServiceProxy = (OrderService) ProxyUtil.newProxyInstance(target);  
        // 调用代理对象的代理方法  
        orderServiceProxy.detail();  
        orderServiceProxy.modify();  
        orderServiceProxy.generate();  
    }  
}
```

```
D:\Java21\bin\java.exe ...
订单信息如下: *****
耗时2544毫秒
订单已修改
耗时1016毫秒
订单已生成
耗时1236毫秒

进程已结束，退出代码为 0
```

CGLIB动态代理

CGLIB既可以代理接口，又可以代理类。底层采用继承的方式实现。所以被代理的目标类不能使用final修饰。使用CGLIB，需要引入它的依赖：

```
<dependency>
  <groupId>cglib</groupId>
  <artifactId>cglib</artifactId>
  <version>3.3.0</version>
</dependency>
```

我们准备一个没有实现接口的类，如下：

- UserService

```
package org.cqipu.edu.service;

public class UserService {

    public void login(){
        System.out.println("用户正在登录系统...");
    }

    public void logout(){
        System.out.println("用户正在退出系统...");
    }
}
```

使用CGLIB在内存中为UserService类生成代理类，并创建对象：

- 客户端程序

```
package org.cqipu.edu.test;
import org.cqipu.edu.service.UserService;
import net.sf.cglib.proxy.Enhancer;

public class Client {
    public static void main(String[] args) {
        // 创建字节码增强器
        Enhancer enhancer = new Enhancer();
        // 告诉cglib要继承哪个类
        enhancer.setSuperclass(UserService.class);
        // 设置回调接口
        enhancer.setCallback(方法拦截器对象);
        // 生成源码，编译class，加载到JVM，并创建代理对象
        UserService userServiceProxy = (UserService)enhancer.create();

        userServiceProxy.login();
        userServiceProxy.logout();
    }
}
```

```
}  
}
```

和JDK动态代理原理差不多，在CGLIB中需要提供的不是InvocationHandler，而是：net.sf.cglib.proxy.MethodInterceptor

编写MethodInterceptor接口实现类：

- TimerMethodInterceptor

```
package org.cqipu.edu.service;  
import net.sf.cglib.proxy.MethodInterceptor;  
import net.sf.cglib.proxy.MethodProxy;  
import java.lang.reflect.Method;  
  
public class TimerMethodInterceptor implements MethodInterceptor {  
    @Override  
    public Object intercept(Object target, Method method, Object[] objects, MethodProxy methodProxy) throws Throwable  
    {  
        return null;  
    }  
}
```

MethodInterceptor接口中有一个方法intercept()，该方法有4个参数：

- 第一个参数：目标对象
- 第二个参数：目标方法
- 第三个参数：目标方法调用时的实参
- 第四个参数：代理方法

在MethodInterceptor的intercept()方法中调用目标以及添加增强：

- TimerMethodInterceptor

```
package org.cqipu.edu.service;  
import net.sf.cglib.proxy.MethodInterceptor;  
import net.sf.cglib.proxy.MethodProxy;  
import java.lang.reflect.Method;  
public class TimerMethodInterceptor implements MethodInterceptor {  
    @Override  
    public Object intercept(Object target, Method method, Object[] objects, MethodProxy methodProxy) throws Throwable  
    {  
        // 前增强  
        long begin = System.currentTimeMillis();  
        // 调用目标  
        Object retValue = methodProxy.invokeSuper(target, objects);  
        // 后增强  
        long end = System.currentTimeMillis();  
        System.out.println("耗时" + (end - begin) + "毫秒");  
        // 一定要返回  
        return retValue;  
    }  
}
```

回调已经写完了，可以修改客户端程序了：

- 客户端程序

```
package org.cqipu.edu.test;
```

```

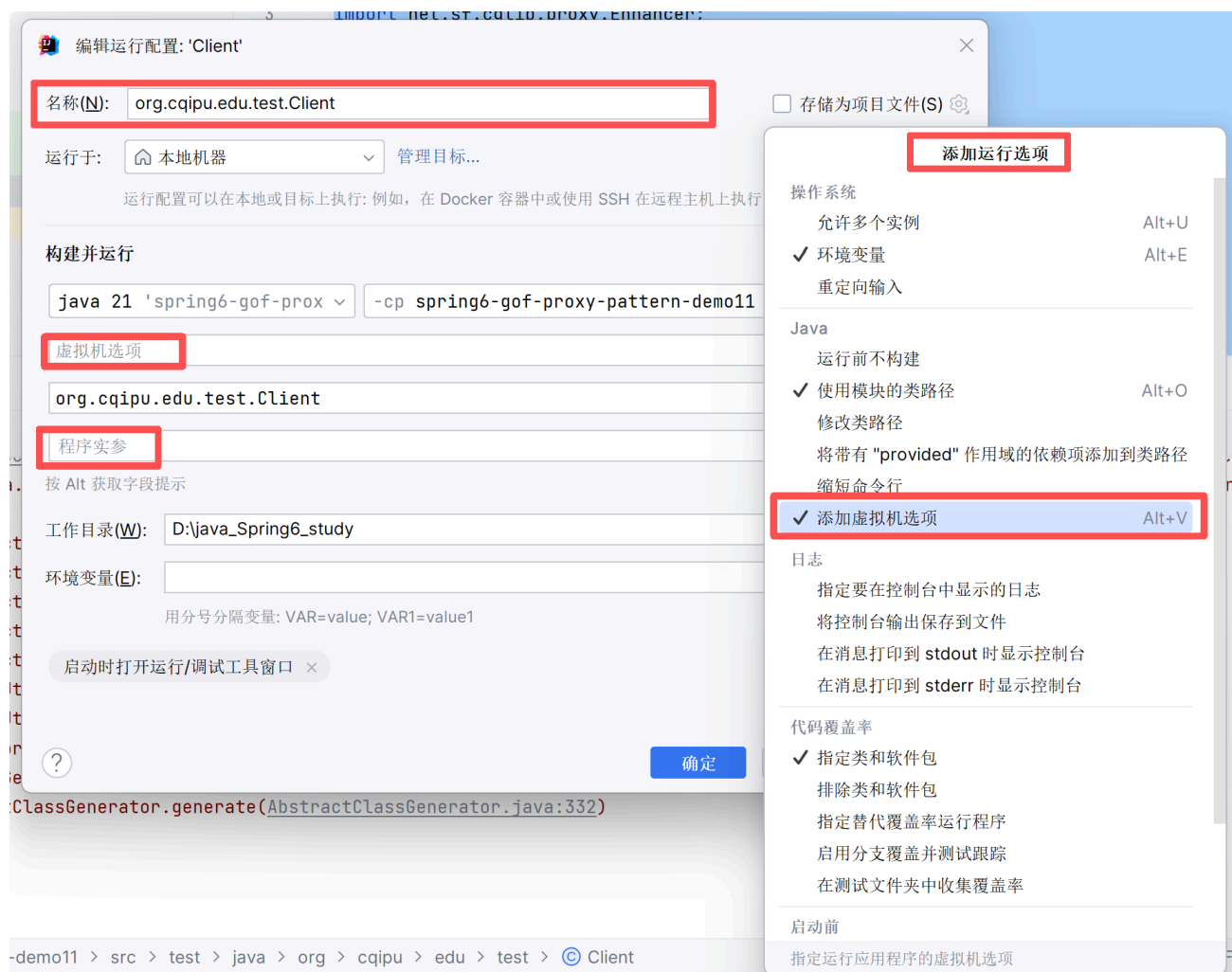
import org.cqipu.edu.service.UserService;
import net.sf.cglib.proxy.Enhancer;
import org.cqipu.edu.service.TimerMethodInterceptor;

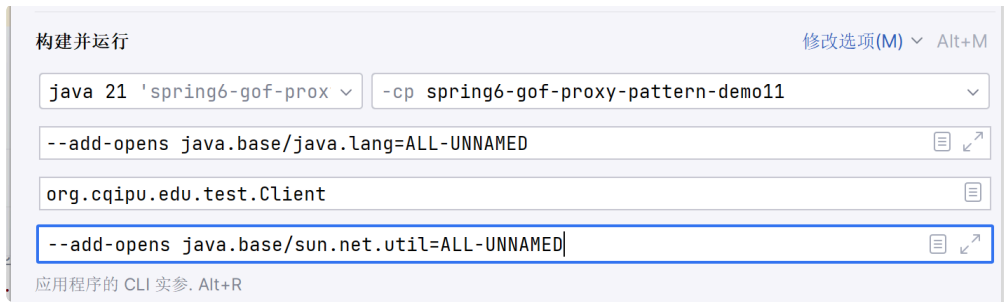
public class Client {
    public static void main(String[] args) {
        // 创建字节码增强器
        Enhancer enhancer = new Enhancer();
        // 告诉cglib要继承哪个类
        enhancer.setSuperclass(UserService.class);
        // 设置回调接口
        enhancer.setCallback(new TimerMethodInterceptor());
        // 生成源码，编译class，加载到JVM，并创建代理对象
        UserService userServiceProxy = (UserService)enhancer.create();

        userServiceProxy.login();
        userServiceProxy.logout();
    }
}

```

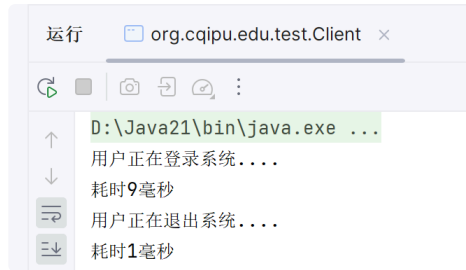
对于高版本的JDK，如果使用CGLIB，需要在启动项中添加两个启动参数：





- --add-opens java.base/java.lang=ALL-UNNAMED
- --add-opens java.base/sun.net.util=ALL-UNNAMED

执行结果：



面向切面编程 AOP

IoC 使软件组件松耦合。AOP 让你能够捕捉系统中经常使用的功能，把它转化成组件。

AOP (Aspect Oriented Programming)：面向切面编程，面向方面编程。(AOP 是一种编程技术)

AOP 是对 OOP 的补充延伸。

AOP 底层使用的就是动态代理来实现的。

Spring 的 AOP 使用的动态代理是：JDK 动态代理 + CGLIB 动态代理技术。Spring 在这两种动态代理中灵活切换，如果是代理接口，会默认使用

JDK 动态代理，如果要代理某个类，这个类没有实现接口，就会切换使用 CGLIB。当然，你也可以强制通过一些配置让 Spring 只使用 CGLIB。

AOP 介绍

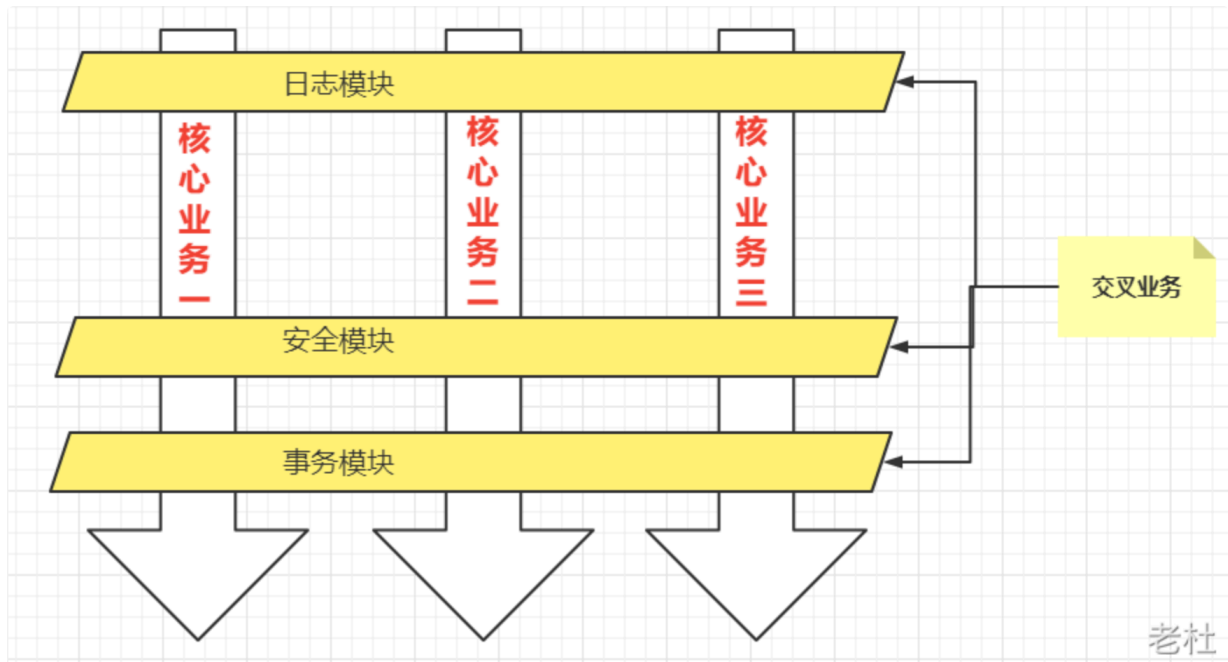
一般一个系统当中都会有一些系统服务，例如：日志、事务管理、安全等。这些系统服务被称为：交叉业务

这些交叉业务几乎是通用的，不管你是做银行账户转账，还是删除用户数据。日志、事务管理、安全，这些都是需要做的。

如果在每一个业务处理过程当中，都掺杂这些交叉业务代码进去的话，存在两方面问题：

- 第一：交叉业务代码在多个业务流程中反复出现，显然这个交叉业务代码没有得到复用。并且修改这些交叉业务代码的话，需要修改多处。
- 第二：程序员无法专注核心业务代码的编写，在编写核心业务代码的同时还需要处理这些交叉业务。

使用 AOP 可以很轻松的解决以上问题。



用一句话总结 AOP：将与核心业务无关的代码独立的抽取出来，形成一个独立的组件，然后以横向交叉的方式应用到业务流程当中的过程被称为 AOP。

AOP 的优点：

- 第一：代码复用性增强。
- 第二：代码易维护。
- 第三：使开发者更关注业务逻辑。

AOP的七大术语

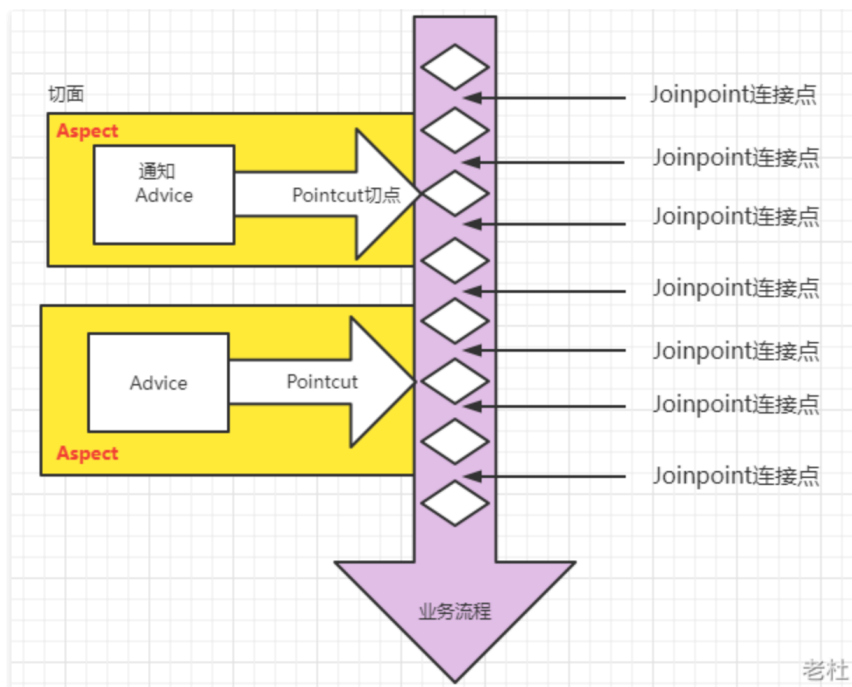
- UserService.java

```
public class UserService{
    public void do1(){
        System.out.println("do 1");
    }
    public void do2(){
        System.out.println("do 2");
    }
    public void do3(){
        System.out.println("do 3");
    }
    public void do4(){
        System.out.println("do 4");
    }
    public void do5(){
        System.out.println("do 5");
    }
    // 核心业务方法
    public void service(){
        do1();
        do2();
        do3();
        do5();
    }
}
```

- 连接点 Joinpoint

- 在程序的整个执行流程中，可以织入切面的位置。方法的执行前后，异常抛出之后等位置。
- **切点 Pointcut**
 - 在程序执行流程中，真正织入切面的方法。（一个切点对应多个连接点）
- **通知 Advice**
 - 通知又叫增强，就是具体你要织入的代码。
 - 通知包括:
 - 前置通知
 - 后置通知
 - 环绕通知
 - 异常通知
 - 最终通知
- **切面 Aspect**
 - 切点 + 通知就是切面。
- **织入 Weaving**
 - 把通知应用到目标对象上的过程。
- **代理对象 Proxy**
 - 一个目标对象被织入通知后产生的新对象。
- **目标对象 Target**
 - 被织入通知的对象。

通过下图，大家可以很好的理解AOP的相关术语:



切点表达式

切点表达式用来定义通知（Advice）往哪些方法上切入。

切入点表达式语法格式：



execution([访问控制权限修饰符] 返回值类型 [全限定类名]方法名(形式参数列表) [异常])

访问控制权限修饰符：

- 可选项。
- 没写，就是 4 个权限都包括。
- 写 public 就表示只包括公开的方法。

返回值类型：

- 必填项。
- * 表示返回值类型任意。

全限定类名：

- 可选项。
- 两个点 “.” 代表当前包以及子包下的所有类。
- 省略时表示所有的类。

方法名：

- 必填项。
- * 表示所有方法。
- set * 表示所有的 set 方法。

形式参数列表：

- 必填项
- () 表示没有参数的方法
- (..) 参数类型和个数随意的方法
- (*) 只有一个参数的方法
- (*, String) 第一个参数类型随意，第二个参数是 String 的。

异常：

- 可选项。
- 省略时表示任意异常类型。

理解以下的切点表达式：

- service包下所有的类中以delete开始的所有方法

```
execution(public * com.powernode.mall.service.*.delete*(..))
```

- mall包下所有的类的所有的方法

```
execution(* com.powernode.mall..*(..))
```

- 所有类的所有方法

```
execution(* *(..))
```

使用 Spring 的 AOP

Spring 对 AOP 的实现包括以下 3 种方式：

- 第一种方式：Spring 框架结合 AspectJ 框架实现的 AOP，基于注解方式。
- 第二种方式：Spring 框架结合 AspectJ 框架实现的 AOP，基于 XML 方式。
- 第三种方式：Spring 框架自己实现的 AOP，基于 XML 配置方式。

实际开发中，都是 Spring+AspectJ 来实现 AOP。所以我们重点学习第一种和第二种方式。

什么是 AspectJ?（Eclipse 组织的一个支持 AOP 的框架。AspectJ 框架是独立于 Spring 框架之外的一个框架，Spring 框架用了 AspectJ）

AspectJ 项目起源于帕洛阿尔托（Palo Alto）研究中心（缩写为 PARC）。该中心由 Xerox 集团资助，Gregor Kiczales 领导，从 1997 年开始致力于 AspectJ 的开发，1998 年第一次发布给外部用户，2001 年发布 1.0 release。为了推动 AspectJ 技术和社团的发展，PARC 在 2003 年 3 月正式将 AspectJ 项目移交给了 Eclipse 组织，因为 AspectJ 的发展和受关注程度大大超出了 PARC 的预期，他们已经无力继续维持它的发展。

准备工作

使用Spring+AspectJ的AOP需要引入的依赖如下：

- pom.xml

```
<!-- spring context 依赖 -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.0.18</version>
</dependency>
<!-- spring aop 依赖 -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-aop</artifactId>
    <version>6.0.18</version>
</dependency>
<!-- spring aspects 依赖 -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-aspects</artifactId>
    <version>6.0.18</version>
</dependency>
```

Spring配置文件中添加context命名空间和aop命名空间

- spring-aspectj-aop-annotation.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:aop="http://www.springframework.org/schema/aop"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context.xsd
        http://www.springframework.org/schema/aop
        http://www.springframework.org/schema/aop/spring-aop.xsd">

</beans>
```

基于 AspectJ 的 AOP 注解式开发

实现步骤

- 第一步：定义目标类以及目标方法
 - 目标类：

```
package org.cqipu.edu.service;

// 目标类
public class OrderService {
    // 目标方法
    public void generate(){
        System.out.println("订单已生成!");
    }
}
```

- 第二步：定义切面类
 - 切面类

```
package org.cqipu.edu.service;
import org.aspectj.lang.annotation.Aspect;

// 切面类
@Aspect
public class MyAspect {
}
```

- 第三步：目标类和切面类都纳入spring bean管理
 - 在目标类OrderService上添加@Component注解。
 - 在切面类MyAspect类上添加@Component注解。
- 第四步：在spring配置文件中添加组建扫描
 - spring-aspectj-aop-annotation.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:context="http://www.springframework.org/schema/context"
        xmlns:aop="http://www.springframework.org/schema/aop"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd
http://www.springframework.org/schema/aop http://www.springframework.org/schema/aop/spring-
aop.xsd">
    <!-- 开启组件扫描-->
    <context:component-scan base-package="org.cqipu.edu.service"/>
</beans>
```

- 第五步：在切面类和目标类中添加通知

```

package org.cqipu.edu.service;
import org.springframework.stereotype.Component;
// 目标类
@Component
public class OrderService {
    // 目标方法
    public void generate(){
        System.out.println("订单已生成!");
    }
}

```

```

package org.cqipu.edu.service;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.stereotype.Component;

// 切面类
@Aspect
@Component
public class MyAspect {
    // 这就是需要增强的代码（通知）
    public void advice(){
        System.out.println("我是一个通知");
    }
}

```

- 第六步：在通知上添加切点表达式
 - 通知+切点=切面

```

package org.cqipu.edu.service;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.stereotype.Component;
import org.aspectj.lang.annotation.Before;

// 切面类
@Aspect
@Component
public class MyAspect {
    // 切点表达式
    @Before("execution(* org.cqipu.edu.service.OrderService.*(..))")
    // 这就是需要增强的代码（通知）
    public void advice(){
        System.out.println("我是一个通知");
    }
}

```

注解@Before表示前置通知。

- 第七步：在spring配置文件中启用自动代理
 - spring-aspectj-aop-annotation.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd
                           http://www.springframework.org/schema/aop http://www.springframework.org/schema/aop/spring-
aop.xsd">
    <!-- 开启组件扫描-->
    <context:component-scan base-package="org.cqipu.edu.service"/>
    <!-- 开启自动代理-->
    <aop:aspectj-autoproxy proxy-target-class="true"/>
</beans>
```

`<aop:aspectj-autoproxy proxy-target-class="true"/>` 开启自动代理之后，凡事带有@Aspect注解的bean都会生成代理对象。

`proxy-target-class="true"` 表示采用cglib动态代理。

`proxy-target-class="false"` 表示采用jdk动态代理。默认值是false。即使写成false，当没有接口的时候，也会自动选择cglib生成代理类。

- 测试程序：



```
package org.cqipu.edu.test;

import org.cqipu.edu.service.OrderService;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class AOPTest {
    @Test
    public void testAOP(){
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-aspectj-aop-
annotation.xml");
        OrderService orderService = applicationContext.getBean("orderService", OrderService.class);
        orderService.generate();
    }
}
```

✓ 测试 已通过: 1共 1 个测试 - 469毫秒

D:\Java21\bin\java.exe ...

我是一个通知

订单已生成!

通知类型

通知类型包括：

- 前置通知：@Before 目标方法执行之前的通知
- 后置通知：@AfterReturning 目标方法执行之后的通知
- 环绕通知：@Around 目标方法之前添加通知，同时目标方法执行之后添加通知。
- 异常通知：@AfterThrowing 发生异常之后执行的通知
- 最终通知：@After 放在finally语句块中的通知

接下来，编写程序来测试这几个通知的执行顺序：

- MyAspect

```
package org.cqipu.edu.service;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.stereotype.Component;

// 切面类
@Component
@Aspect
public class MyAspect {

    @Around("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void aroundAdvice(ProceedingJoinPoint proceedingJoinPoint) throws Throwable {
        System.out.println("环绕通知开始");
        // 执行目标方法。
        proceedingJoinPoint.proceed();
        System.out.println("环绕通知结束");
    }

    @Before("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void beforeAdvice(){
        System.out.println("前置通知");
    }

    @AfterReturning("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void afterReturningAdvice(){
        System.out.println("后置通知");
    }

    @AfterThrowing("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void afterThrowingAdvice(){
        System.out.println("异常通知");
    }

    @After("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void afterAdvice(){
        System.out.println("最终通知");
    }
}
```

- 目标类和目标方法

```
package org.cqipu.edu.service;
import org.springframework.stereotype.Component;

// 目标类
@Component
public class OrderService {
    // 目标方法
    public void generate(){
        System.out.println("订单已生成!");
    }
}
```

- 测试程序

```
package org.cqipu.edu.test;

import org.cqipu.edu.service.OrderService;
```

```
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class AOPTest {
    @Test
    public void testAOP(){
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-aspectj-aop-annotation.xml");
        OrderService orderService = applicationContext.getBean("orderService", OrderService.class);
        orderService.generate();
    }
}
```

执行结果：

✓ 测试 已通过: 1共 1 个测试 - 417毫秒

D:\Java21\bin\java.exe ...

环绕通知开始

前置通知

订单已生成!

后置通知

最终通知

环绕通知结束

通过上面的执行结果就可以判断他们的执行顺序了，这里不再赘述。

结果中没有异常通知，这是因为目标程序执行过程中没有发生异常。我们尝试让目标方法发生异常：

- 让目标方法执行过程中发生异常

```
package org.cqipu.edu.service;
import org.springframework.stereotype.Component;

// 目标类
@Component
public class OrderService {
    // 目标方法
    public void generate(){
        System.out.println("订单已生成!");
        if (1 == 1) {
            throw new RuntimeException("模拟异常发生");
        }
    }
}
```

再次执行

❗ 测试 失败: 1共 1 个测试 - 455毫秒

D:\Java21\bin\java.exe ...

环绕通知开始

前置通知

订单已生成!

异常通知

最终通知

java.lang.RuntimeException: 模拟异常发生

通过测试得知，当发生异常之后，最终通知也会执行，因为最终通知 `@After` 会出现在 `finally` 语句块中。

出现异常之后，**后置通知** 和 **环绕通知的结束部分** 不会执行。

切面的先后顺序

我们知道，业务流程当中不一定只有一个切面，可能有的切面控制事务，有的记录日志，有的进行安全控制，如果多个切面的话，顺序如何控制：**可以使用 `@Order` 注解来标识切面类，为 `@Order` 注解的 `value` 指定一个整数型的数字，数字越小，优先级越高。**

- 前提，取消异常抛出

```
package org.cqipu.edu.service;
import org.springframework.stereotype.Component;

// 目标类
@Component
public class OrderService {
    // 目标方法
    public void generate(){
        System.out.println("订单已生成!");
        if (1 == 1) {
            throw new RuntimeException("模拟异常发生");
        }
    }
}
```

- 再定义一个切面类，如下：
 - 另一个切面类，并设置优先级

```
package org.cqipu.edu.service;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Aspect
@Component
@Order(1) // 设置优先级
public class YourAspect {

    @Around("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void aroundAdvice(ProceedingJoinPoint proceedingJoinPoint) throws Throwable {
        System.out.println("YourAspect 环绕通知开始");
        // 执行目标方法。
        proceedingJoinPoint.proceed();
        System.out.println("YourAspect 环绕通知结束");
    }

    @Before("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void beforeAdvice(){
        System.out.println("YourAspect 前置通知");
    }

    @AfterReturning("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void afterReturningAdvice(){
        System.out.println("YourAspect 后置通知");
    }

    @AfterThrowing("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void afterThrowingAdvice(){
        System.out.println("YourAspect 异常通知");
    }

    @After("execution(* org.cqipu.edu.service.OrderService.*(..))")
```

```

    public void afterAdvice(){
        System.out.println("YourAspect 最终通知");
    }
}

```

- 设置切面类MyAspect的优先级

```

package org.cqipu.edu.service;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.stereotype.Component;
import org.springframework.core.annotation.Order;

// 切面类
@Component
@Aspect
@Order(2) // 设置优先级
public class MyAspect {

    @Around("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void aroundAdvice(ProceedingJoinPoint proceedingJoinPoint) throws Throwable {
        System.out.println("环绕通知开始");
        // 执行目标方法。
        proceedingJoinPoint.proceed();
        System.out.println("环绕通知结束");
    }

    @Before("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void beforeAdvice(){
        System.out.println("前置通知");
    }

    @AfterReturning("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void afterReturningAdvice(){
        System.out.println("后置通知");
    }

    @AfterThrowing("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void afterThrowingAdvice(){
        System.out.println("异常通知");
    }

    @After("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void afterAdvice(){
        System.out.println("最终通知");
    }
}

```

测试结果

✓ 测试 已通过: 1共 1 个测试 - 416毫秒

D:\Java21\bin\java.exe ...

YourAspect环绕通知开始

YourAspect前置通知

环绕通知开始

前置通知

订单已生成!

后置通知

最终通知

环绕通知结束

YourAspect后置通知

YourAspect最终通知

YourAspect环绕通知结束

通过修改@Order注解的整数值来切换顺序，执行测试程序：

```
@Aspect
@Component
@Order(2) // 设置优先级
public class YourAspect {
```

```
// 切面类
@Component
@Aspect
@Order(1) // 设置优先级
public class MyAspect {
```

✓ 测试 已通过: 1共 1 个测试 - 462毫秒

D:\Java21\bin\java.exe ...

环绕通知开始

前置通知

YourAspect环绕通知开始

YourAspect前置通知

订单已生成!

YourAspect后置通知

YourAspect最终通知

YourAspect环绕通知结束

后置通知

最终通知

环绕通知结束

优化使用切点表达式

观看以下代码中的切点表达式：

```
package org.cqipu.edu.service;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.stereotype.Component;
import org.springframework.core.annotation.Order;

// 切面类
@Component
@Aspect
@Order(2) // 设置优先级
public class MyAspect {

    @Around("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void aroundAdvice(ProceedingJoinPoint proceedingJoinPoint) throws Throwable {
        System.out.println("环绕通知开始");
```

```

        // 执行目标方法。
        proceedingJoinPoint.proceed();
        System.out.println("环绕通知结束");
    }

    @Before("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void beforeAdvice(){
        System.out.println("前置通知");
    }

    @AfterReturning("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void afterReturningAdvice(){
        System.out.println("后置通知");
    }

    @AfterThrowing("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void afterThrowingAdvice(){
        System.out.println("异常通知");
    }

    @After("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void afterAdvice(){
        System.out.println("最终通知");
    }
}

```

缺点是：

- 第一：切点表达式重复写了多次，没有得到复用。
- 第二：如果要修改切点表达式，需要修改多处，难维护。

可以这样做：将切点表达式单独的定义出来，在需要的位置引入即可。如下：

```

package org.cqipu.edu.service;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.*;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

// 切面类
@Component
@Aspect
@Order(2)
public class MyAspect {

    @Pointcut("execution(* org.cqipu.edu.service.OrderService.*(..))")
    public void pointcut(){}

    @Around("pointcut()")
    public void aroundAdvice(ProceedingJoinPoint proceedingJoinPoint) throws Throwable {
        System.out.println("环绕通知开始");
        // 执行目标方法。
        proceedingJoinPoint.proceed();
        System.out.println("环绕通知结束");
    }

    @Before("pointcut()")
    public void beforeAdvice(){
        System.out.println("前置通知");
    }

    @AfterReturning("pointcut()")
    public void afterReturningAdvice(){

```

```

        System.out.println("后置通知");
    }

    @AfterThrowing("pointcut()")
    public void afterThrowingAdvice(){
        System.out.println("异常通知");
    }

    @After("pointcut()")
    public void afterAdvice(){
        System.out.println("最终通知");
    }
}

```

使用 `@Pointcut` 注解来定义独立的切点表达式。

注意这个 `@Pointcut` 注解标注的方法随意，只是起到一个能够让 `@Pointcut` 注解编写的位置。

执行测试程序：

✓ 测试 已通过: 1共 1 个测试 - 442毫秒

D:\Java21\bin\java.exe ...

YourAspect环绕通知开始

YourAspect前置通知

环绕通知开始

前置通知

订单已生成！

后置通知

最终通知

环绕通知结束

YourAspect后置通知

YourAspect最终通知

YourAspect环绕通知结束

全注解式开发AOP

就是编写一个类，在这个类上面使用大量注解来代替spring的配置文件，spring配置文件消失了，如下：

```

package org.cqipu.edu.service;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableAspectJAutoProxy;

@Configuration
@ComponentScan("org.cqipu.edu.service")
@EnableAspectJAutoProxy(proxyTargetClass = true)
public class Spring6Configuration {
}

```

测试程序也变化了：

```

package org.cqipu.edu.test;

import org.cqipu.edu.service.OrderService;
import org.cqipu.edu.service.Spring6Configuration;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

```

```

public class AOPTest {
    @Test
    public void testAOPWithAllAnnotation(){
        ApplicationContext applicationContext = new AnnotationConfigApplicationContext(Spring6Configuration.class);
        OrderService orderService = applicationContext.getBean("orderService", OrderService.class);
        orderService.generate();
    }
}

```

执行结果

```

D:\Java21\bin\java.exe ...
YourAspect环绕通知开始
YourAspect前置通知
环绕通知开始
前置通知
订单已生成!
后置通知
最终通知
环绕通知结束
YourAspect后置通知
YourAspect最终通知
YourAspect环绕通知结束

```

基于XML配置方式的AOP（了解）

- 第一步：编写目标类
 - 不添加@Component注解

```

package org.cqipu.edu.service;

// 目标类
public class VipService {
    public void add(){
        System.out.println("保存vip信息。");
    }
}

```

- 第二步：编写切面类，并且编写通知
 - 不添加@Component注解

```

package org.cqipu.edu.service;
import org.aspectj.lang.ProceedingJoinPoint;

// 负责计时的切面类
public class TimerAspect {

    public void time(ProceedingJoinPoint proceedingJoinPoint) throws Throwable {
        Long begin = System.currentTimeMillis();
        // 执行目标
        proceedingJoinPoint.proceed();
        Long end = System.currentTimeMillis();
        System.out.println("耗时" + (end - begin) + "毫秒");
    }
}

```

- 第三步：编写spring配置文件
 - spring-aop-xml.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd
                           http://www.springframework.org/schema/aop http://www.springframework.org/schema/aop/spring-
aop.xsd">

    <!-- 纳入spring bean 管理-->
    <bean id="vipService" class="org.cqipu.edu.service.VipService"/>
    <bean id="timerAspect" class="org.cqipu.edu.service.TimerAspect"/>

    <!-- aop 配置-->
    <aop:config>
        <!-- 切点表达式-->
        <aop:pointcut id="p" expression="execution(* org.cqipu.edu.service.VipService.*(..))"/>
        <!-- 切面-->
        <aop:aspect ref="timerAspect">
            <!-- 切面=通知 + 切点-->
            <aop:around method="time" pointcut-ref="p"/>
        </aop:aspect>
    </aop:config>
</beans>
```

测试程序



```
package org.cqipu.edu.test;

import org.cqipu.edu.service.VipService;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class AOPTest3 {

    @Test
    public void testAOPXml(){
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-aop-xml.xml");
        VipService vipService = applicationContext.getBean("vipService", VipService.class);
        vipService.add();
    }
}
```

执行结果

✓ 测试 已通过: 1 共 1 个测试 - 361毫秒

D:\Java21\bin\java.exe ...

保存vip信息。

耗时0毫秒

AOP的实际案例：事务处理

项目中的事务控制是在所难免的。在一个业务流程当中，可能需要多条DML语句共同完成，为了保证数据的安全，这多条DML语句要么同时成功，要么同时失败。这就需要添加事务控制的代码。例如以下伪代码：

- 伪代码

```
class 业务类1{
    public void 业务方法1(){
        try{
            // 开启事务
            startTransaction();

            // 执行核心业务逻辑
            step1();
            step2();
            step3();
            ....

            // 提交事务
            commitTransaction();
        }catch(Exception e){
            // 回滚事务
            rollbackTransaction();
        }
    }
    public void 业务方法2(){
        try{
            // 开启事务
            startTransaction();

            // 执行核心业务逻辑
            step1();
            step2();
            step3();
            ....

            // 提交事务
            commitTransaction();
        }catch(Exception e){
            // 回滚事务
            rollbackTransaction();
        }
    }
    public void 业务方法3(){
        try{
            // 开启事务
            startTransaction();

            // 执行核心业务逻辑
            step1();
            step2();
            step3();
            ....

            // 提交事务
            commitTransaction();
        }catch(Exception e){
            // 回滚事务
            rollbackTransaction();
        }
    }
}

class 业务类2{
    public void 业务方法1(){
        try{
            // 开启事务
            startTransaction();
```

```

        // 执行核心业务逻辑
        step1();
        step2();
        step3();
        ....

        // 提交事务
        commitTransaction();
    }catch(Exception e){
        // 回滚事务
        rollbackTransaction();
    }
}

public void 业务方法2(){
    try{
        // 开启事务
        startTransaction();

        // 执行核心业务逻辑
        step1();
        step2();
        step3();
        ....

        // 提交事务
        commitTransaction();
    }catch(Exception e){
        // 回滚事务
        rollbackTransaction();
    }
}

public void 业务方法3(){
    try{
        // 开启事务
        startTransaction();

        // 执行核心业务逻辑
        step1();
        step2();
        step3();
        ....

        // 提交事务
        commitTransaction();
    }catch(Exception e){
        // 回滚事务
        rollbackTransaction();
    }
}

}
//.....

```

可以看到，这些业务类中的每一个业务方法都是需要控制事务的，而控制事务的代码又是固定的格式，都是：

```

try{
    // 开启事务
    startTransaction();

    // 执行核心业务逻辑
    //.....

    // 提交事务
    commitTransaction();
}catch(Exception e){
    // 回滚事务
    rollbackTransaction();
}

```

这个控制事务的代码就是和业务逻辑没有关系的“交叉业务”。以上伪代码当中可以看到这些交叉业务的代码没有得到复用，并且如果这些交叉业务代码需要修改，那必然需要修改多处，难维护，怎么解决？可以采用AOP思想解决。可以把以上控制事务的代码作为环绕通知，切入到目标类的方法当中。接下来我们做一下这件事，有两个业务类，如下：

```

package org.cqipu.edu.service;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableAspectJAutoProxy;

@Configuration
@ComponentScan("org.cqipu.edu.biz")
@EnableAspectJAutoProxy(proxyTargetClass = true)
public class Spring6Configuration {
}

```

- 银行账户的业务类

```

package org.cqipu.edu.biz;
import org.springframework.stereotype.Component;

@Component
// 业务类
public class AccountService {
    // 转账业务方法
    public void transfer(){
        System.out.println("正在进行银行账户转账");
    }
    // 取款业务方法
    public void withdraw(){
        System.out.println("正在进行取款操作");
    }
}

```

- 订单业务类

```

package org.cqipu.edu.biz;
import org.springframework.stereotype.Component;

@Component
// 业务类
public class OrderService {
    // 生成订单
    public void generate(){

```



```

        System.out.println("正在生成订单");
    }
    // 取消订单
    public void cancel(){
        System.out.println("正在取消订单");
    }
}

```

注意，以上两个业务类已经纳入spring bean的管理，因为都添加了@Component注解。

接下来我们给以上两个业务类的4个方法添加事务控制代码，使用AOP来完成

- 事务切面类

```

package org.cqipu.edu.biz;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.stereotype.Component;

@Aspect
@Component
// 事务切面类
public class TransactionAspect {

    @Around("execution(* org.cqipu.edu.biz..*(..))")
    public void aroundAdvice(ProceedingJoinPoint proceedingJoinPoint){
        try {
            System.out.println("开启事务");
            // 执行目标
            proceedingJoinPoint.proceed();
            System.out.println("提交事务");
        } catch (Throwable e) {
            System.out.println("回滚事务");
        }
    }
}

```

你看，这个事务控制代码是不是只需要写一次就行了，并且修改起来也没有成本。编写测试程序：

```

package org.cqipu.edu.test;

import org.cqipu.edu.biz.AccountService;
import org.cqipu.edu.biz.OrderService;
import org.cqipu.edu.service.Spring6Configuration;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class AOPTest2 {
    @Test
    public void testTransaction(){
        ApplicationContext applicationContext = new AnnotationConfigApplicationContext(Spring6Configuration.class);
        OrderService orderService = applicationContext.getBean("orderService", OrderService.class);
        AccountService accountService = applicationContext.getBean("accountService", AccountService.class);
        // 生成订单
        orderService.generate();
        // 取消订单
        orderService.cancel();
        // 转账
        accountService.transfer();
        // 取款
    }
}

```

```

        accountService.withdraw();
    }
}

```

✓ 测试 已通过: 1共 1 个测试 - 347毫秒

D:\Java21\bin\java.exe ...

开启事务
 正在生成订单
 提交事务
 开启事务
 正在取消订单
 提交事务
 开启事务
 正在进行银行账户转账
 提交事务
 开启事务
 正在进行取款操作
 提交事务

AOP 的实际案例：安全日志

需求是这样的：项目开发结束了，已经上线了。运行正常。客户提出了新的需求：凡事在系统中进行修改操作的，删除操作的，新增操作的，都要把这个人记录下来。因为这几个操作是属于危险行为。例如有业务类和业务方法：

- 用户业务类

```

package org.cqipu.edu.biz;
import org.springframework.stereotype.Component;

@Component
// 用户业务
public class UserService {
    public void getUser(){
        System.out.println("获取用户信息");
    }
    public void saveUser(){
        System.out.println("保存用户");
    }
    public void deleteUser(){
        System.out.println("删除用户");
    }
    public void modifyUser(){
        System.out.println("修改用户");
    }
}

```

- 商品业务类

```

package org.cqipu.edu.biz;
import org.springframework.stereotype.Component;

// 商品业务类
@Component
public class ProductService {
    public void getProduct(){
        System.out.println("获取商品信息");
    }
}

```

```

    }
    public void saveProduct(){
        System.out.println("保存商品");
    }
    public void deleteProduct(){
        System.out.println("删除商品");
    }
    public void modifyProduct(){
        System.out.println("修改商品");
    }
}

```

注意：已经添加了@Component注解。

接下来我们使用aop来解决上面的需求：编写一个负责安全的切面类

- 负责安全的切面类

```

package org.cqipu.edu.biz;
import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;
import org.springframework.stereotype.Component;

@Component
@Aspect
public class SecurityAspect {

    @Pointcut("execution(* org.cqipu.edu.biz..save*(..))")
    public void savePointcut(){}

    @Pointcut("execution(* org.cqipu.edu.biz..delete*(..))")
    public void deletePointcut(){}

    @Pointcut("execution(* org.cqipu.edu.biz..modify*(..))")
    public void modifyPointcut(){}

    @Before("savePointcut() || deletePointcut() || modifyPointcut()")
    public void beforeAdvice(JoinPoint joinpoint){
        System.out.println("XXX操作员正在操作"+joinpoint.getSignature().getName()+"方法");
    }
}

```

- 测试程序

```

package org.cqipu.edu.test;

import org.cqipu.edu.biz.AccountService;
import org.cqipu.edu.biz.OrderService;
import org.cqipu.edu.biz.ProductService;
import org.cqipu.edu.biz.UserService;
import org.cqipu.edu.service.Spring6Configuration;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class AOPTest2 {

    @Test
    public void testSecurity(){
        ApplicationContext applicationContext = new AnnotationConfigApplicationContext(Spring6Configuration.class);
        UserService userService = applicationContext.getBean("userService", UserService.class);
    }
}

```

```

    ProductService productService = applicationContext.getBean("productService", ProductService.class);
    userService.getUser();
    userService.saveUser();
    userService.deleteUser();
    userService.modifyUser();
    productService.getProduct();
    productService.saveProduct();
    productService.deleteProduct();
    productService.modifyProduct();
}
}

```

运行之前，把 **TransactionAspect** 事务切面类的注解注释掉

```

//@Aspect
//@Component
// 事务切面类
0 个用法
public class TransactionAspect {

```

```

D:\Java21\bin\java.exe ...
获取用户信息
XXX操作员正在操作saveUser方法
保存用户
XXX操作员正在操作deleteUser方法
删除用户
XXX操作员正在操作modifyUser方法
修改用户
获取商品信息
XXX操作员正在操作saveProduct方法
保存商品
XXX操作员正在操作deleteProduct方法
删除商品
XXX操作员正在操作modifyProduct方法
修改商品

```

Spring对事务的支持

事务概述

- 什么是事务
 - 在一个业务流程当中，通常需要多条 DML（insert delete update）语句共同联合才能完成，这多条 DML 语句必须同时成功，或者同时失败，这样才能保证数据的安全。
 - 多条 DML 要么同时成功，要么同时失败，这叫做事务。
 - 事务：Transaction（tx）
- 事务的四个处理过程：
 - 第一步：开启事务（start transaction）
 - 第二步：执行核心业务代码
 - 第三步：提交事务（如果核心业务处理过程中没有出现异常）（commit transaction）
 - 第四步：回滚事务（如果核心业务处理过程中出现异常）（rollback transaction）
- 事务的四个特性：
 - A 原子性：事务是最小的工作单元，不可再分。
 - C 一致性：事务要求要么同时成功，要么同时失败。事务前和事务后的总量不变。

- I 隔离性：事务和事务之间因为有隔离性，才可以保证互不干扰。
- D 持久性：持久性是事务结束的标志。

引入事务场景

以银行账户转账为例学习事务。两个账户 act-001 和 act-002。act-001 账户向 act-002 账户转账 10000，必须同时成功，或者同时失败。（一个减成功，一个加成功，这两条 update 语句必须同时成功，或同时失败。）

连接数据库的技术采用 Spring 框架的 JdbcTemplate。

采用三层架构搭建：



（注：表现层、业务逻辑层、数据访问层共同组成 Web 后端）

- 模块名：spring6-tx-bank-demo13
- pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.cqipu.edu</groupId>
    <artifactId>spring6-tx-bank-demo13</artifactId>
    <version>1.0-SNAPSHOT</version>

    <!-- 仓库 -->
    <repositories>
        <!-- spring 里程碑版本的仓库 -->
        <repository>
            <id>repository.spring.milestone</id>
            <name>Spring Milestone Repository</name>
            <url>https://repo.spring.io/milestone</url>
        </repository>
    </repositories>

    <!-- 依赖 -->
    <dependencies>
        <!-- spring context -->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
```

```
        <version>6.0.18</version>
    </dependency>
    <!--spring jdbc-->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-jdbc</artifactId>
        <version>6.0.18</version>
    </dependency>
    <!--mysql 驱动-->
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.30</version>
    </dependency>
    <!-- 德鲁伊连接池-->
    <dependency>
        <groupId>com.alibaba</groupId>
        <artifactId>druid</artifactId>
        <version>1.2.13</version>
    </dependency>
    <!--@Resource注解-->
    <dependency>
        <groupId>jakarta.annotation</groupId>
        <artifactId>jakarta.annotation-api</artifactId>
        <version>2.1.1</version>
    </dependency>
    <!--junit-->
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>

<properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
</properties>
</project>
```

第一步：准备数据库表

在spring6数据库创建 **t_act** 数据表

表结构

名称	类型	长度	小数点	不是 null	虚拟	键	注释
actno	varchar	255		☒	☐	 1	账号
balance	double			☐	☐		余额

表数据

表配置文件		开始事务	
actno	balance		
varchar(255)	# double		
act-001	50000		
act-002	0		

第二步：创建包结构

- org.cqipu.edu.pojo
- org.cqipu.edu.service
- org.cqipu.edu.service.impl
- org.cqipu.edu.dao
- org.cqipu.edu.dao.impl

第三步：准备POJO类

- Account.java

```
package org.cqipu.edu.pojo;
public class Account {
    private String actno;
    private Double balance;

    @Override
    public String toString() {
        return "Account{" +
            "actno='" + actno + '\'' +
            ", balance=" + balance +
            '}';
    }

    public Account() {
    }

    public Account(String actno, Double balance) {
        this.actno = actno;
        this.balance = balance;
    }

    public String getActno() {
        return actno;
    }

    public void setActno(String actno) {
        this.actno = actno;
    }

    public Double getBalance() {
        return balance;
    }

    public void setBalance(Double balance) {
        this.balance = balance;
    }
}
```

第四步：编写持久层

- AccountDaoImpl.java

```
package org.cqipu.edu.dao.impl;

import org.cqipu.edu.dao.AccountDao;
import org.cqipu.edu.pojo.Account;
import jakarta.annotation.Resource;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
```

```

import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.stereotype.Repository;

@Repository("accountDao")
public class AccountDaoImpl implements AccountDao {

    @Resource(name = "jdbcTemplate")
    private JdbcTemplate jdbcTemplate;

    @Override
    public Account selectByActno(String actno) {
        String sql = "select actno, balance from t_act where actno = ?";
        Account account = jdbcTemplate.queryForObject(sql, new BeanPropertyRowMapper<>(Account.class), actno);
        return account;
    }

    @Override
    public int update(Account act) {
        String sql = "update t_act set balance = ? where actno = ?";
        int count = jdbcTemplate.update(sql, act.getBalance(), act.getActno());
        return count;
    }
}

```

第五步：编写业务层

- AccountService.java

```

package org.cqipu.edu.service;

public interface AccountService {

    /**
     * 转账
     * @param fromActno
     * @param toActno
     * @param money
     */
    void transfer(String fromActno, String toActno, double money);
}

```

- AccountServiceImpl.java

```

package org.cqipu.edu.service.impl;

import org.cqipu.edu.dao.AccountDao;
import org.cqipu.edu.pojo.Account;
import org.cqipu.edu.service.AccountService;
import jakarta.annotation.Resource;
import org.springframework.stereotype.Service;

@Service("accountService")
public class AccountServiceImpl implements AccountService {

    @Resource(name = "accountDao")
    private AccountDao accountDao;

    @Override
    public void transfer(String fromActno, String toActno, double money) {

```



```

// 查询账户余额是否充足
Account fromAct = accountDao.selectByActno(fromActno);
if (fromAct.getBalance() < money) {
    throw new RuntimeException("账户余额不足");
}
// 余额充足, 开始转账
Account toAct = accountDao.selectByActno(toActno);
fromAct.setBalance(fromAct.getBalance() - money);
toAct.setBalance(toAct.getBalance() + money);
int count = accountDao.update(fromAct);
count += accountDao.update(toAct);
if (count != 2) {
    throw new RuntimeException("转账失败, 请联系银行");
}
}
}

```

第六步：编写Spring配置文件

- spring.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd">

    <context:component-scan base-package="org.cqipu.edu"/>

    <bean id="dataSource" class="com.alibaba.druid.pool.DruidDataSource">
        <property name="driverClassName" value="com.mysql.cj.jdbc.Driver"/>
        <property name="url" value="jdbc:mysql://localhost:3306/spring6"/>
        <property name="username" value="root"/>
        <property name="password" value="20060329"/>
    </bean>

    <bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
        <property name="dataSource" ref="dataSource"/>
    </bean>

</beans>

```

第七步：编写表示层（测试程序）

```

package org.cqipu.edu.test;
import org.cqipu.edu.service.AccountService;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class BankTest {
    @Test
    public void testTransfer(){
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
        AccountService accountService = applicationContext.getBean("accountService", AccountService.class);
        try {
            accountService.transfer("act-001", "act-002", 10000);
            System.out.println("转账成功");
        }
    }
}

```

```
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}  
}
```

执行结果

✓ 测试 已通过: 1共 1 个测试 - 1秒 219毫秒

D:\Java21\bin\java.exe ...

4月 15, 2026 8:07:49 下午 com.alibaba.druid.support.logging.JakartaCommonsLoggingImpl info

信息: {dataSource-1} initied

转账成功

数据变化

对象 t_act @spring6 (本地MySQL) - 表

actno	balance
act-001	40000
act-002	10000

模拟异常

```
package org.cqipu.edu.service.impl;  
import org.cqipu.edu.dao.AccountDao;  
import org.cqipu.edu.pojo.Account;  
import org.cqipu.edu.service.AccountService;  
import jakarta.annotation.Resource;  
import org.springframework.stereotype.Service;  
  
@Service("accountService")  
public class AccountServiceImpl implements AccountService {  
  
    @Resource(name = "accountDao")  
    private AccountDao accountDao;  
  
    @Override  
    public void transfer(String fromActno, String toActno, double money) {  
        // 查询账户余额是否充足  
        Account fromAct = accountDao.selectByActno(fromActno);  
        if (fromAct.getBalance() < money) {  
            throw new RuntimeException("账户余额不足");  
        }  
        // 余额充足, 开始转账  
        Account toAct = accountDao.selectByActno(toActno);  
        fromAct.setBalance(fromAct.getBalance() - money);  
        toAct.setBalance(toAct.getBalance() + money);  
        int count = accountDao.update(fromAct);  
  
        // 模拟异常  
        String s = null;  
        s.toString();  
  
        count += accountDao.update(toAct);  
        if (count != 2) {  
            throw new RuntimeException("转账失败, 请联系银行");  
        }  
    }  
}
```

执行结果

```
✓ 测试 已通过: 1共 1 个测试 - 840毫秒
D:\Java21\bin\java.exe ...
4月 15, 2026 8:11:00 下午 com.alibaba.druid.support.logging.JakartaCommonsLoggingImpl info
信息: {dataSource-1} inited
java.lang.NullPointerException Create breakpoint : Cannot invoke "String.toString()" because "s" is null
    at org.cqipu.edu.service.impl.AccountServiceImpl.transfer(AccountServiceImpl.java:29)
    at org.cqipu.edu.test.BankTest.testTransfer(BankTest.java:13) <25 个内部行>
```

数据表中的变化

对象 t_act @spring6 (本地MySQL) - 表	
表配置文件	开始事务 单元格编辑器 筛选
actno	balance
varchar(255)	double
act-001	30000
act-002	10000

丢了一万

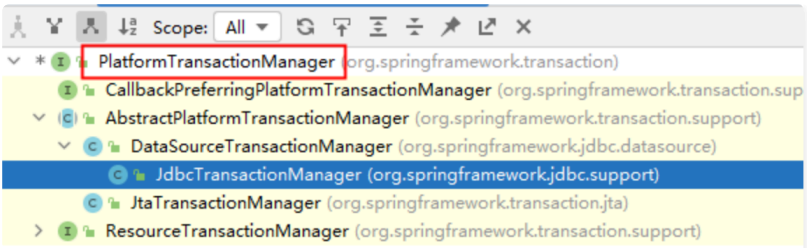
Spring 对事务的支持

Spring 实现事务的两种方式

- 编程式事务
 - 通过编写代码的方式来管理事务。
- 声明式事务
 - 基于注解方式
 - 基于 XML 配置方式

Spring 事务管理 API

Spring 对事务的管理底层实现方式是基于 AOP 实现的。采用 AOP 的方式进行了封装。所以 Spring 专门针对事务开发了一套 API，API 的核心接口如下：



PlatformTransactionManager 接口：spring 事务管理器的核心接口。在 Spring6 中它有两个实现：

- **DataSourceTransactionManager**：支持 JdbcTemplate、MyBatis、Hibernate 等事务管理。
- **JtaTransactionManager**：支持分布式事务管理。

如果要在 Spring6 中使用 JdbcTemplate，就要使用 **DataSourceTransactionManager** 来管理事务。（Spring 内置写好了，可以直接用。）

声明式事务之注解实现方式

- 第一步：在spring配置文件中配置事务管理器。



```
<bean id="transactionManager" class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
    <property name="dataSource" ref="dataSource"/>
</bean>
```

- 第二步：在spring配置文件中引入tx命名空间。



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:tx="http://www.springframework.org/schema/tx"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context http://www.springframework.org/schema/context/spring-
context.xsd
        http://www.springframework.org/schema/tx http://www.springframework.org/schema/tx/spring-tx.xsd">

    <!-- 组件扫描：自动扫描@Component、@Service、@Repository等注解 -->
    <context:component-scan base-package="org.cqipu.edu"/>

    <!-- Druid数据源配置 -->
    <bean id="dataSource" class="com.alibaba.druid.pool.DruidDataSource">
        <property name="driverClassName" value="com.mysql.cj.jdbc.Driver"/>
        <property name="url" value="jdbc:mysql://localhost:3306/spring6"/>
        <property name="username" value="root"/>
        <property name="password" value="20060329"/>
    </bean>

    <!-- JdbcTemplate配置 -->
    <bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
        <property name="dataSource" ref="dataSource"/>
    </bean>

    <!-- 事务管理器配置 -->
    <bean id="transactionManager" class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
        <property name="dataSource" ref="dataSource"/>
    </bean>

</beans>
```

- 第三步：在spring配置文件中配置“事务注解驱动器”，开始注解的方式控制事务。



```
<tx:annotation-driven transaction-manager="transactionManager"/>
```

- 第四步：在service类上或方法上添加@Transactional注解
 - 在类上添加该注解，该类中所有的方法都有事务。在某个方法上添加该注解，表示只有这个方法使用事务。



```
package org.cqipu.edu.service.impl;
import org.cqipu.edu.dao.AccountDao;
import org.cqipu.edu.pojo.Account;
import org.cqipu.edu.service.AccountService;
import jakarta.annotation.Resource;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
```

```

@Service("accountService")
@Transactional
public class AccountServiceImpl implements AccountService {

    @Resource(name = "accountDao")
    private AccountDao accountDao;

    @Override
    public void transfer(String fromActno, String toActno, double money) {
        // 查询账户余额是否充足
        Account fromAct = accountDao.selectByActno(fromActno);
        if (fromAct.getBalance() < money) {
            throw new RuntimeException("账户余额不足");
        }
        // 余额充足, 开始转账
        Account toAct = accountDao.selectByActno(toActno);
        fromAct.setBalance(fromAct.getBalance() - money);
        toAct.setBalance(toAct.getBalance() + money);
        int count = accountDao.update(fromAct);

        // 模拟异常
        String s = null;
        s.toString();

        count += accountDao.update(toAct);
        if (count != 2) {
            throw new RuntimeException("转账失败, 请联系银行");
        }
    }
}

```

当前数据表的数据

对象 t_act @spring6 (本地MySQL) - 表	
表配置文件	开始事务
单元格编辑器	筛
actno varchar(255)	balance double
act-001	30000
act-002	10000

执行测试

✓ 测试已通过: 1 共 1 个测试 - 954毫秒

D:\Java21\bin\java.exe ...

4月 15, 2026 8:24:46 下午 com.alibaba.druid.support.logging.JakartaCommonsLoggingImpl info

信息: {dataSource-1} initied

java.lang.NullPointerException Create breakpoint : Cannot invoke "String.toString()" because "s" is null

虽然出现异常了, 但是数据表的数据没有变

对象 t_act @spring6 (本地MySQL) - 表	
表配置文件	开始事务
单元格编辑器	筛
actno varchar(255)	balance double
act-001	30000
act-002	10000

通过测试, 发现数据没有变化, 事务起作用了。

事务属性

事务属性包括哪些

```
public @interface Transactional {  
    @AliasFor("transactionManager")  
    String value() default "";  
  
    @AliasFor("value")  
    String transactionManager() default "";  
  
    String[] label() default {};  
  
    Propagation propagation() default Propagation.REQUIRED;  
  
    Isolation isolation() default Isolation.DEFAULT;  
  
    int timeout() default -1;  
  
    String timeoutString() default "";  
  
    boolean readOnly() default false;  
  
    Class<? extends Throwable>[] rollbackFor() default {};  
  
    String[] rollbackForClassName() default {};  
  
    Class<? extends Throwable>[] noRollbackFor() default {};  
  
    String[] noRollbackForClassName() default {};  
}
```

事务中的重点属性:

- 事务传播行为
- 事务隔离级别
- 事务超时
- 只读事务
- 设置出现哪些异常回滚事务
- 设置出现哪些异常不回滚事务

事务传播行为

什么是事务的传播行为?

在 service 类中有 a () 方法和 b () 方法, a () 方法上有事务, b () 方法上也有事务, 当 a () 方法执行过程中调用了 b () 方法, 事务是如何传递的? 合并到一个事务里? 还是开启一个新的事务? 这就是事务传播行为。

事务传播行为在 spring 框架中被定义为枚举类型:

```

6 package org.springframework.transaction.annotation;
7
8 public enum Propagation {
9     REQUIRED( value: 0),
10    SUPPORTS( value: 1),
11    MANDATORY( value: 2),
12    REQUIRES_NEW( value: 3),
13    NOT_SUPPORTED( value: 4),
14    NEVER( value: 5),
15    NESTED( value: 6);

```

老

一共有七种传播行为：

- **REQUIRED**：支持当前事务，如果不存在就新建一个 (默认)【没有就新建，有就加入】
- **SUPPORTS**：支持当前事务，如果当前没有事务，就以非事务方式执行【有就加入，没有就不管了】
- **MANDATORY**：必须运行在一个事务中，如果当前没有事务正在发生，将抛出一个异常【有就加入，没有就抛异常】
- **REQUIRES_NEW**：开启一个新的事务，如果一个事务已经存在，则将这个存在的事务挂起【不管有没有，直接开启一个新事务，开启的新事务和之前的事务不存在嵌套关系，之前事务被挂起】
- **NOT_SUPPORTED**：以非事务方式运行，如果有事务存在，挂起当前事务【不支持事务，存在就挂起】
- **NEVER**：以非事务方式运行，如果有事务存在，抛出异常【不支持事务，存在就抛异常】
- **NESTED**：如果当前正有一个事务在进行中，则该方法应当运行在一个嵌套式事务中。被嵌套的事务可以独立于外层事务进行提交或回滚。如果外层事务不存在，行为就像 REQUIRED 一样。【有事务的话，就在这个事务里再嵌套一个完全独立的事务，嵌套的事务可以独立的提交和回滚。没有事务就和 REQUIRED 一样。】

在代码中设置事务的传播行为：

```

@Transactional(propagation = Propagation.REQUIRED)

```

可以编写程序测试一下传播行为：

- 1号service

```

@Transactional(propagation = Propagation.REQUIRED)
public void save(Account act) {

    // 这里调用dao的insert方法。
    accountDao.insert(act); // 保存act-003账户

    // 创建账户对象
    Account act2 = new Account("act-004", 1000.0);
    try {
        accountService.save(act2); // 保存act-004账户
    } catch (Exception e) {

    }
    // 继续往后进行我当前1号事务自己的事儿。
}

```

- 2号service

```
● ● ●
@Override
//@Transactional(propagation = Propagation.REQUIRED)
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void save(Account act) {
    accountDao.insert(act);
    // 模拟异常
    String s = null;
    s.toString();

    // 事儿没有处理完，这个大括号当中的后续也许还有其他的DML语句。
}
```

一定要集成 Log4j2 日志框架，在日志信息中可以看到更加详细的信息。

事务隔离级别

事务隔离级别类似于教室 A 和教室 B 之间的那道墙，隔离级别越高表示墙体越厚。隔音效果越好。

数据库中读取数据存在的三大问题：（三大读问题）

- 脏读：读取到没有提交到数据库的数据，叫做脏读。
- 不可重复读：在同一个事务当中，第一次和第二次读取的数据不一样。
- 幻读：读到的数据是假的。

事务隔离级别包括四个级别：

- 读未提交：READ_UNCOMMITTED
 - 这种隔离级别，存在脏读问题，所谓的脏读 (dirty read) 表示能够读取到其它事务未提交的数据。
- 读提交：READ_COMMITTED
 - 解决了脏读问题，其它事务提交之后才能读到，但存在不可重复读问题。
- 可重复读：REPEATABLE_READ
 - 解决了不可重复读，可以达到可重复读效果，只要当前事务不结束，读取到的数据一直都是一样的。但存在幻读问题。
- 序列化：SERIALIZABLE
 - 解决了幻读问题，事务排队执行。不支持并发。

大家可以通过一个表格来记忆：

隔离级别	脏读	不可重复读
读未提交	有	有
读提交	无	有
可重复读	无	无
序列化	无	无

在 Spring 代码中如何设置隔离级别？

隔离级别在 spring 中以枚举类型存在：


```
public enum Isolation {
    DEFAULT( value: -1),
    READ_UNCOMMITTED( value: 1),
    READ_COMMITTED( value: 2),
    REPEATABLE_READ( value: 4),
    SERIALIZABLE( value: 8);
}
```

```
@Transactional(isolation = Isolation.READ_COMMITTED)
```

测试事务隔离级别：READ_UNCOMMITTED 和 READ_COMMITTED

怎么测试：一个service负责插入，一个service负责查询。负责插入的service要模拟延迟。

- 修改 AccountDao 接口
 - 新增了 insert 方法

```
package org.cqipu.edu.dao;

import org.cqipu.edu.pojo.Account;
public interface AccountDao {

    /**
     * 根据账号查询余额
     * @param actno
     * @return
     */
    Account selectByActno(String actno);

    /**
     * 更新账户
     * @param act
     * @return
     */
    int update(Account act);

    int insert(Account act);
}
```

- AccountDaoImpl.java

```
package org.cqipu.edu.dao.impl;

import org.cqipu.edu.dao.AccountDao;
import org.cqipu.edu.pojo.Account;
import jakarta.annotation.Resource;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository("accountDao")
public class AccountDaoImpl implements AccountDao {

    @Resource(name = "jdbcTemplate")
    private JdbcTemplate jdbcTemplate;
```

```

@Override
public Account selectByActno(String actno) {
    String sql = "select actno, balance from t_act where actno = ?";
    List<Account> accountList = jdbcTemplate.query(sql, new BeanPropertyRowMapper<>(Account.class), actno);
    return accountList.isEmpty() ? null : accountList.get(0);
}

@Override
public int update(Account act) {
    String sql = "update t_act set balance = ? where actno = ?";
    int count = jdbcTemplate.update(sql, act.getBalance(), act.getActno());
    return count;
}

@Override
public int insert(Account act) {
    String sql = "insert into t_act(actno, balance) values(?, ?)";
    return jdbcTemplate.update(sql, act.getActno(), act.getBalance());
}
}

```

- IsolationService1.java

```

package org.cqipu.edu.service.impl;

import org.cqipu.edu.dao.AccountDao;
import org.cqipu.edu.pojo.Account;
import jakarta.annotation.Resource;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Isolation;
import org.springframework.transaction.annotation.Transactional;

@Service("i1")
public class IsolationService1 {

    @Resource(name = "accountDao")
    private AccountDao accountDao;

    @Transactional(isolation = Isolation.READ_COMMITTED)
    public void getByActno(String actno) {
        Account account = accountDao.selectByActno(actno);
        if (account == null) {
            System.out.println("查询的账户不存在: " + actno);
        } else {
            System.out.println("查询到的账户信息: " + account);
        }
    }
}

```

- IsolationService2.java

```

package org.cqipu.edu.service.impl;
import org.cqipu.edu.dao.AccountDao;
import org.cqipu.edu.pojo.Account;
import jakarta.annotation.Resource;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service("i2")
public class IsolationService2 {

    @Resource(name = "accountDao")

```

```

private AccountDao accountDao;

// 2号
// 负责insert
@Transactional
public void save(Account act) {
    accountDao.insert(act);
    // 睡眠一会
    try {
        Thread.sleep(1000 * 20);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
}

```

测试程序

```

● ● ●
@Test
public void testIsolation1(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
    IsolationService1 i1 = applicationContext.getBean("i1", IsolationService1.class);
    i1.getByActno("act-004");
}

@Test
public void testIsolation2(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
    IsolationService2 i2 = applicationContext.getBean("i2", IsolationService2.class);
    Account act = new Account("act-004", 1000.0);
    i2.save(act);
}

```

第一个测试结果

✓ 测试 已通过: 1共 1 个测试 - 960毫秒

D:\Java21\bin\java.exe ...

4月 15, 2026 8:53:23 下午 com.alibaba.druid.support.logging.JakartaCommonsLoggingImpl info
 信息: {dataSource-1} initied
 查询的账户不存在: act-004

第二个测试结果

✓ 测试 已通过: 1共 1 个测试 - 20秒 921毫秒

D:\Java21\bin\java.exe ...

4月 15, 2026 8:54:18 下午 com.alibaba.druid.support.logging.JakartaCommonsLoggingImpl info
 信息: {dataSource-1} initied

事务超时

代码如下:

```

● ● ●
@Transactional(timeout = 10)

```

以上代码表示设置事务的超时时间为 10 秒。

表示超过 10 秒如果该事务中所有的 DML 语句还没有执行完毕的话，最终结果会选择回滚。

默认值 - 1，表示没有时间限制。

这里有个坑，事务的超时时间指的是哪段时间？

在当前事务当中，最后一条 DML 语句执行之前的时间。如果最后一条 DML 语句后面有很多业务逻辑，这些业务代码执行的时间不被计入超时时间。

- 以下代码的超时不会被计入超时时间

```
● ● ●
@Transactional(timeout = 10) // 设置事务超时时间为10秒。
public void save(Account act) {
    accountDao.insert(act);
    // 睡眠一会
    try {
        Thread.sleep(1000 * 15);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
```

- 以下代码超时时间会被计入超时时间

```
● ● ●
@Transactional(timeout = 10) // 设置事务超时时间为10秒。
public void save(Account act) {
    // 睡眠一会
    try {
        Thread.sleep(1000 * 15);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    accountDao.insert(act);
}
```

当然，如果想让整个方法的所有代码都计入超时时间的话，可以在方法最后一行添加一行无关紧要的 DML 语句。

只读事务

代码如下：

```
● ● ●
@Transactional(readOnly = true)
```

将当前事务设置为只读事务，在该事务执行过程中只允许 select 语句执行，delete insert update 均不可执行。

该特性的作用是：启动 spring 的优化策略，提高 select 语句执行效率。

如果该事务中确实没有增删改操作，建议设置为只读事务。

设置哪些异常回滚事务

代码如下：

```
● ● ●
@Transactional(rollbackFor = RuntimeException.class)
```

表示只有发生 RuntimeException 异常或该异常的子类异常才回滚。

设置哪些异常不回滚事务

代码如下：

```
● ● ●
@Transactional(noRollbackFor = NullPointerException.class)
```

设置哪些异常不回滚事务

代码如下：

```
● ● ●
@Transactional(noRollbackFor = NullPointerException.class)
```

表示发生 `NullPointerException` 或该异常的子类异常不回滚，其他异常则回滚。

事务的全注解式开发

编写一个类来代替配置文件，代码如下：

```
● ● ●
package org.cqipu.edu.bank;

import com.alibaba.druid.pool.DruidDataSource;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.sql.DataSource;

@Configuration
@ComponentScan("org.cqipu.edu")
@EnableTransactionManagement
public class Spring6Config {

    @Bean
    public DataSource getDataSource(){
        DruidDataSource dataSource = new DruidDataSource();
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/spring6");
        dataSource.setUsername("root");
        dataSource.setPassword("20060329");
        return dataSource;
    }

    @Bean(name = "jdbcTemplate")
    public JdbcTemplate getJdbcTemplate(DataSource dataSource){
        JdbcTemplate jdbcTemplate = new JdbcTemplate();
        jdbcTemplate.setDataSource(dataSource);
        return jdbcTemplate;
    }

    @Bean
    public DataSourceTransactionManager getDataSourceTransactionManager(DataSource dataSource){
        DataSourceTransactionManager dataSourceTransactionManager = new DataSourceTransactionManager();
        dataSourceTransactionManager.setDataSource(dataSource);
        return dataSourceTransactionManager;
    }
}
```

```
}
```

测试程序

```
@Test
```

```
public void testNoXml(){
    ApplicationContext applicationContext = new AnnotationConfigApplicationContext(Spring6Config.class);
    AccountService accountService = applicationContext.getBean("accountService", AccountService.class);
    try {
        accountService.transfer("act-001", "act-002", 10000);
        System.out.println("转账成功");
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

✓ 测试 已通过: 1共 1 个测试 - 1秒 129毫秒

```
D:\Java21\bin\java.exe ...
```

```
4月 20, 2026 2:08:51 下午 com.alibaba.druid.support.logging.JakartaCommonsLoggingImpl info
```

```
信息: {dataSource-1} inited
```

```
java.lang.NullPointerException Create breakpoint : Cannot invoke "String.toString()" because "s" is null
```

```
> at org.cqipu.edu.service.impl.AccountServiceImpl.transfer(AccountServiceImpl.java:31) <2 个内部行>
```

```
at org.springframework.aop.support.AopUtils.invokeJoinpointUsingReflection(AopUtils.java:343)
```

```
at org.springframework.aop.framework.ReflectiveMethodInvocation.invokeJoinpoint(ReflectiveMethodInvocation.java:196)
```

对象 t_act @spring6 (本地MySQL) - 表	
表配置文件	开始事务 单元格编辑器
actno	balance
varchar(255)	# double
act-001	30000
act-002	10000
act-004	1000

声明式事务之 XML 实现方式

配置步骤：

- 第一步：配置事务管理器
- 第二步：配置通知
- 第三步：配置切面

记得添加 aspectj 的依赖：

- pom.xml

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-aspects</artifactId>
    <version>6.1.12</version>
</dependency>
```

Spring 配置文件如下：

记得添加 aop 的命名空间。

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
```

```

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xmlns:tx="http://www.springframework.org/schema/tx"
xmlns:aop="http://www.springframework.org/schema/aop"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd
http://www.springframework.org/schema/tx http://www.springframework.org/schema/tx/spring-
tx.xsd
http://www.springframework.org/schema/aop http://www.springframework.org/schema/aop/spring-
aop.xsd">

<context:component-scan base-package="org.cqipu.edu"/>

<bean id="dataSource" class="com.alibaba.druid.pool.DruidDataSource">
  <property name="driverClassName" value="com.mysql.cj.jdbc.Driver"/>
  <property name="url" value="jdbc:mysql://localhost:3306/spring6"/>
  <property name="username" value="root"/>
  <property name="password" value="20060329"/>
</bean>

<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
  <property name="dataSource" ref="dataSource"/>
</bean>

<!--配置事务管理器-->
<bean id="transactionManager" class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
  <property name="dataSource" ref="dataSource"/>
</bean>

<!--配置通知-->
<tx:advice id="txAdvice" transaction-manager="transactionManager">
  <tx:attributes>
    <tx:method name="save*" propagation="REQUIRED" rollback-for="java.lang.Throwable"/>
    <tx:method name="del*" propagation="REQUIRED" rollback-for="java.lang.Throwable"/>
    <tx:method name="update*" propagation="REQUIRED" rollback-for="java.lang.Throwable"/>
    <tx:method name="transfer*" propagation="REQUIRED" rollback-for="java.lang.Throwable"/>
  </tx:attributes>
</tx:advice>

<!--配置切面-->
<aop:config>
  <aop:pointcut id="txPointcut" expression="execution(* org.cqipu.edu.service..*(..))"/>
  <!--切面 = 通知 + 切点-->
  <aop:advisor advice-ref="txAdvice" pointcut-ref="txPointcut"/>
</aop:config>

</beans>

```

将 AccountServiceImpl 类上的 @Transactional 注解删除。
编写测试程序：

```

@Test
public void testTransferXml(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring2.xml");
    AccountService accountService = applicationContext.getBean("accountService", AccountService.class);
    try {
        accountService.transfer("act-001", "act-002", 10000);
        System.out.println("转账成功");
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

测试结果

```

✓ 测试 已通过: 1共 1 个测试 - 1秒 331毫秒

D:\Java21\bin\java.exe ...
4月 20, 2026 2:19:46 下午 org.springframework.core.LocalVariableTableParameterNameDiscoverer inspectClass
警告: Using deprecated '-debug' fallback for parameter name resolution. Compile the affected code with '-parameters' instead or avoid ...
4月 20, 2026 2:19:46 下午 com.alibaba.druid.support.logging.JakartaCommonsLoggingImpl info
信息: {dataSource-1} inited

```

数据表记录

actno	balance
act-001	30000
act-002	10000

可以看到配置的XML已经起作用了

Spring6 整合 JUnit5

Spring 对 JUnit4 的支持

准备工作:

- pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.cqipu.edu</groupId>
    <artifactId>spring6-junit5-demo14</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <!--spring context 稳定版-->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>6.1.12</version>
        </dependency>
        <!--spring-test 整合JUnit 稳定版-->
        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-test</artifactId>

```



```

        <version>6.1.12</version>
        <scope>test</scope>
    </dependency>
    <!--junit4 依赖-->
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>

<properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
</properties>

</project>

```

- 声明Bean
 - User.java

```

package org.cqipu.edu.bean;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class User {

    @Value("张三")
    private String name;

    @Override
    public String toString() {
        return "User{" +
            "name='" + name + '\'' +
            '}';
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public User() {
    }

    public User(String name) {
        this.name = name;
    }
}

```

- spring.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd">
    <context:component-scan base-package="org.cqipu.edu.bean"/>
</beans>
```

单元测试



```
package org.cqipu.edu.test;

import org.cqipu.edu.bean.User;
import org.junit.Test;
import org.junit.runner.RunWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.test.context.ContextConfiguration;
import org.springframework.test.context.junit4.SpringJUnit4ClassRunner;

@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration("classpath:spring.xml")
public class SpringJUnit4Test {

    @Autowired
    private User user;

    @Test
    public void testUser(){
        System.out.println(user.getName());
    }
}
```

执行结果：

✓ 测试 已通过: 1 共 1 个测试 - 4 毫秒

D:\Java21\bin\java.exe ...

张三

Spring 提供的方便主要是这几个注解：



```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration("classpath:spring.xml")
```

在单元测试类上使用这两个注解之后，在单元测试类中的属性上可以使用 `@Autowired`。比较方便。

Spring 对 JUnit5 的支持

引入 JUnit5 的依赖，Spring 对 JUnit 支持的依赖还是：spring-test，如下：



```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```

<modelVersion>4.0.0</modelVersion>

<groupId>org.cqipu.edu</groupId>
<artifactId>spring6-junit5-demo14</artifactId>
<version>1.0-SNAPSHOT</version>

<dependencies>
  <!--spring context-->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.1.12</version>
  </dependency>
  <!--spring-test 整合JUnit-->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-test</artifactId>
    <version>6.1.12</version>
    <scope>test</scope>
  </dependency>
  <!--junit5 依赖-->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.9.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>

<properties>
  <maven.compiler.source>21</maven.compiler.source>
  <maven.compiler.target>21</maven.compiler.target>
</properties>

</project>

```

单元测试类

```

package org.cqipu.edu.test;

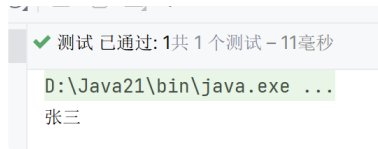
import org.cqipu.edu.bean.User;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.test.context.ContextConfiguration;
import org.springframework.test.context.junit.jupiter.SpringExtension;

@ExtendWith(SpringExtension.class)
@ContextConfiguration("classpath:spring.xml")
public class SpringJUnit5Test {

    @Autowired
    private User user;

    @Test
    public void testUser(){
        System.out.println(user.getName());
    }
}

```



在 JUnit5 当中，可以使用 Spring 提供的以下两个注解，标注到单元测试类上，这样在类当中就可以使用 `@Autowired` 注解了。

```
@ExtendWith(SpringExtension.class)
@ContextConfiguration("classpath:spring.xml")
```

Spring6 集成 MyBatis3.5

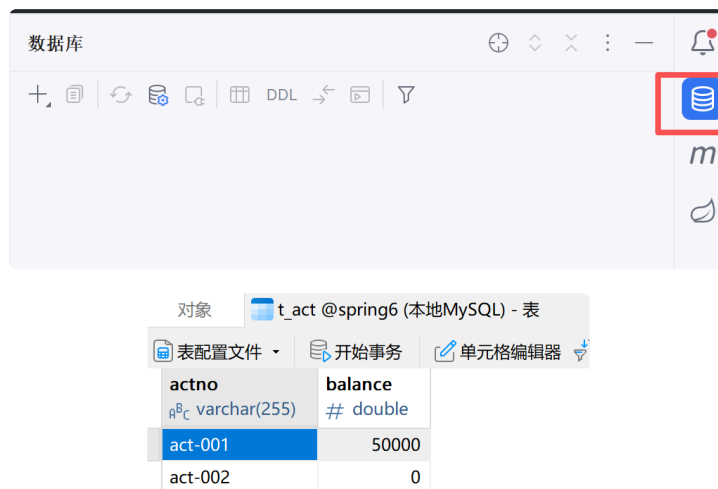
实现步骤

- 第一步：准备数据库表
 - 使用t_act表（账户表）
- 第二步：IDEA 中创建一个模块，并引入依赖
 - spring-context
 - spring-jdbc
 - mysql驱动
 - mybatis
 - mybatis-spring：mybatis 提供的与 spring 框架集成的依赖
 - 德鲁伊连接池
 - junit
- 第三步：基于三层架构实现，所以提前创建好所有的包
 - `org.cqipu.edu.mapper`
 - `org.cqipu.edu.service`
 - `org.cqipu.edu.service.impl`
 - `org.cqipu.edu.pojo`
- 第四步：编写 pojo
 - Account，属性私有化，提供公开的setter getter和toString。
- 第五步：编写 mapper 接口
 - AccountMapper接口，定义方法
- 第六步：编写 mapper 配置文件
 - 在配置文件中配置命名空间，以及每一个方法对应的 sql。
- 第七步：编写 service 接口和 service 接口实现类
 - AccountService
 - AccountServiceImpl
- 第八步：编写jdbc.properties配置文件
 - 数据库连接池相关信息
- 第九步：编写mybatis-config.xml配置文件
 - 该文件可以没有，大部分的配置可以转移到 spring 配置文件中。
 - 如果遇到 mybatis 相关的系统级配置，还是需要这个文件。
- 第十步：编写spring.xml配置文件
 - 组件扫描

- 引入外部的属性文件
- 数据源
- SqlSessionFactoryBean配置
 - 注入 mybatis 核心配置文件路径
 - 指定别名包
 - 注入数据源
- Mapper 扫描配置器
 - 指定扫描的包
- 事务管理器DataSourceTransactionManager
 - 注入数据源
- 启用事务注解
 - 注入事务管理器
- 第十一步：编写测试程序，并添加事务，进行测试

具体实现

- 第一步：准备数据库表
- 连接数据库的工具很多，除了之前我们使用的 navicat for mysql 之外，也可以使用 IDEA 工具自带的 DataBase 插件。可以根据下图提示自行配置：



- 第二步：IDEA中创建一个模块，并引入依赖
 - pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.cqipu.edu</groupId>
  <artifactId>spring6-mybatis3.5-demo15</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <!-- Spring6 核心容器-->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>6.1.12</version>
    </dependency>
```

```

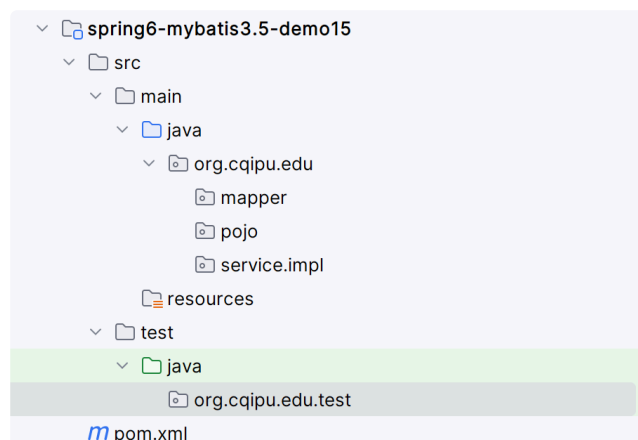
<!-- Spring6 JDBC-->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>6.1.12</version>
</dependency>
<!-- MySQL 驱动-->
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.33</version>
</dependency>
<!-- MyBatis-->
<dependency>
    <groupId>org.mybatis</groupId>
    <artifactId>mybatis</artifactId>
    <version>3.5.11</version>
</dependency>
<!-- MyBatis-Spring 整合-->
<dependency>
    <groupId>org.mybatis</groupId>
    <artifactId>mybatis-spring</artifactId>
    <version>3.0.3</version>
</dependency>
<!-- 德鲁伊连接池-->
<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>druid</artifactId>
    <version>1.2.13</version>
</dependency>
<!-- JUnit4 测试-->
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
</dependency>
</dependencies>

<properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
</properties>

</project>

```

- 第三步：基于三层架构实现，所以提前创建好所有的包
 - 包结构如下：



- 第四步：编写 pojo
 - Account.java

```
package org.cqipu.edu.pojo;
public class Account {
    private String actno;
    private Double balance;

    @Override
    public String toString() {
        return "Account{" +
            "actno='" + actno + '\'' +
            ", balance=" + balance +
            '}';
    }

    public Account() {
    }

    public Account(String actno, Double balance) {
        this.actno = actno;
        this.balance = balance;
    }

    public String getActno() {
        return actno;
    }

    public void setActno(String actno) {
        this.actno = actno;
    }

    public Double getBalance() {
        return balance;
    }

    public void setBalance(Double balance) {
        this.balance = balance;
    }
}
```

- 第五步：编写mapper接口
 - AccountMapper.java

```
package org.cqipu.edu.mapper;
import org.cqipu.edu.pojo.Account;
import java.util.List;

public interface AccountMapper {

    /**
     * 保存账户
     * @param account
     * @return
     */
    int insert(Account account);

    /**
     * 根据账号删除账户
     * @param actno
     * @return
     */
}
```

```

    */
    int deleteByActno(String actno);

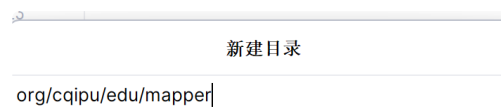
    /**
     * 修改账户
     * @param account
     * @return
     */
    int update(Account account);

    /**
     * 根据账号查询账户
     * @param actno
     * @return
     */
    Account selectByActno(String actno);

    /**
     * 获取所有账户
     * @return
     */
    List<Account> selectAll();
}

```

- 第六步：编写 mapper 配置文件
 - 一定要注意，按照下图提示创建这个目录。注意是斜杠不是点儿。在resources目录下新建，并且要和 Mapper 接口包对应上。



如果接口叫做AccountMapper，配置文件必须是AccountMapper.xml

- AccountMapper.xml

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="org.cqipu.edu.mapper.AccountMapper">
    <insert id="insert">
        insert into t_act values(#{actno}, #{balance})
    </insert>
    <delete id="deleteByActno">
        delete from t_act where actno = #{actno}
    </delete>
    <update id="update">
        update t_act set balance = #{balance} where actno = #{actno}
    </update>
    <select id="selectByActno" resultType="Account">
        select * from t_act where actno = #{actno}
    </select>
    <select id="selectAll" resultType="Account">
        select * from t_act
    </select>
</mapper>

```

- 第七步：编写 service 接口和 service 接口实现类

注意编写的 service 实现类纳入 IoC 容器管理:

- AccountService接口



```
package org.cqipu.edu.service;
import org.cqipu.edu.pojo.Account;
import java.util.List;

public interface AccountService {
    /**
     * 开户
     * @param act
     * @return
     */
    int save(Account act);

    /**
     * 根据账号销户
     * @param actno
     * @return
     */
    int deleteByActno(String actno);

    /**
     * 修改账户
     * @param act
     * @return
     */
    int update(Account act);

    /**
     * 根据账号获取账户
     * @param actno
     * @return
     */
    Account getByActno(String actno);

    /**
     * 获取所有账户
     * @return
     */
    List<Account> getAll();

    /**
     * 转账
     * @param fromActno
     * @param toActno
     * @param money
     */
    void transfer(String fromActno, String toActno, double money);
}
```

- AccountServiceImpl, 注入AccountMapper



```
package org.cqipu.edu.service.impl;
import org.cqipu.edu.mapper.AccountMapper;
import org.cqipu.edu.pojo.Account;
import org.cqipu.edu.service.AccountService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Transactional
@Service("accountService")
```

```

public class AccountServiceImpl implements AccountService {

    @Autowired
    private AccountMapper accountMapper;

    @Override
    public int save(Account act) {
        return accountMapper.insert(act);
    }

    @Override
    public int deleteByActno(String actno) {
        return accountMapper.deleteByActno(actno);
    }

    @Override
    public int update(Account act) {
        return accountMapper.update(act);
    }

    @Override
    public Account getByActno(String actno) {
        return accountMapper.selectByActno(actno);
    }

    @Override
    public List<Account> getAll() {
        return accountMapper.selectAll();
    }

    @Override
    public void transfer(String fromActno, String toActno, double money) {
        Account fromAct = accountMapper.selectByActno(fromActno);
        if (fromAct.getBalance() < money) {
            throw new RuntimeException("余额不足");
        }
        Account toAct = accountMapper.selectByActno(toActno);
        fromAct.setBalance(fromAct.getBalance() - money);
        toAct.setBalance(toAct.getBalance() + money);
        int count = accountMapper.update(fromAct);
        count += accountMapper.update(toAct);
        if (count != 2) {
            throw new RuntimeException("转账失败");
        }
    }
}

```

- 第八步：编写 `jdbc.properties` 配置文件
 - 放在类的根路径下
 - `jdbc.properties`

```

jdbc.driver=com.mysql.cj.jdbc.Driver
jdbc.url=jdbc:mysql://localhost:3306/spring6
jdbc.username=root
jdbc.password=20060329

```

- 第九步：编写 `mybatis-config.xml` 配置文件
 - 放在类的根路径下，只开启日志，其他配置到 `spring.xml` 中。
 - `mybatis-config.xml`

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE configuration
    PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <settings>
        <setting name="LogImpl" value="STDOUT_LOGGING"/>
    </settings>
</configuration>

```

- 第十步：编写spring.xml配置文件
 - 注意：当你在spring.xml文件中直接写标签内容时，IDEA 会自动给你添加命名空间
 - spring.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:tx="http://www.springframework.org/schema/tx"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd http://www.springframework.org/schema/context
https://www.springframework.org/schema/context/spring-context.xsd http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx.xsd">

    <!-- 组件扫描 -->
    <context:component-scan base-package="org.cqipu.edu"/>

    <!-- 外部属性配置文件 -->
    <context:property-placeholder location="jdbc.properties"/>

    <!-- 数据源 -->
    <bean id="dataSource" class="com.alibaba.druid.pool.DruidDataSource">
        <property name="driverClassName" value="${jdbc.driver}"/>
        <property name="url" value="${jdbc.url}"/>
        <property name="username" value="${jdbc.username}"/>
        <property name="password" value="${jdbc.password}"/>
    </bean>

    <!-- SqlSessionFactoryBean -->
    <bean class="org.mybatis.spring.SqlSessionFactoryBean">
        <!-- mybatis 核心配置文件路径 -->
        <property name="configLocation" value="mybatis-config.xml"/>
        <!-- 注入数据源 -->
        <property name="dataSource" ref="dataSource"/>
        <!-- 起别名 -->
        <property name="typeAliasesPackage" value="org.cqipu.edu.pojo"/>
    </bean>

    <!-- Mapper 扫描器 -->
    <bean class="org.mybatis.spring.mapper.MapperScannerConfigurer">
        <property name="basePackage" value="org.cqipu.edu.mapper"/>
    </bean>

    <!-- 事务管理器 -->
    <bean id="txManager" class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
        <property name="dataSource" ref="dataSource"/>
    </bean>

    <!-- 开启事务注解 -->
    <tx:annotation-driven transaction-manager="txManager"/>

```

</beans>

- 第十一步：编写测试程序，并添加事务，进行测试



```
package org.cqipu.edu.test;
import org.cqipu.edu.service.AccountService;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class SMTest {

    @Test
    public void testSM(){
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
        AccountService accountService = applicationContext.getBean("accountService", AccountService.class);
        try {
            accountService.transfer("act-001", "act-002", 10000.0);
            System.out.println("转账成功");
        } catch (Exception e) {
            e.printStackTrace();
            System.out.println("转账失败");
        }
    }
}
```

✓ 测试已通过: 1共 1 个测试 - 1秒 720毫秒

D:\Java21\bin\java.exe ...

Logging initialized using 'class org.apache.ibatis.logging.stdout.StdOutImpl' adapter.

4月 20, 2026 4:04:45 下午 com.alibaba.druid.support.logging.JakartaCommonsLoggingImpl info

信息: {dataSource-1} inited

Creating a new SqlSession

Registering transaction synchronization for SqlSession [org.apache.ibatis.session.defaults.DefaultSqlSession@3a0807b7]

JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@40bffbca] will be managed by Spring

==> Preparing: select * from t_act where actno = ?

==> Parameters: act-001(String)

<== Columns: actno, balance

对象 t_act @spring6 (本地MySQL) - 表	
表配置文件	开始事务
单元格编辑器	
actno	balance
varchar(255)	# double
act-001	40000
act-002	10000

Spring 配置文件的 import

spring 配置文件有多个，并且可以在 spring 的核心配置文件中使⤵import进行引入，我们可以将组件扫描单独定义到一个配置文件中，如下：

- common.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd http://www.springframework.org/schema/context
https://www.springframework.org/schema/context/spring-context.xsd">

    <!-- 组件扫描 -->
    <context:component-scan base-package="org.cqipu.edu"/>

</beans>
```

然后在核心配置文件中引入：

- spring.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:tx="http://www.springframework.org/schema/tx"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd http://www.springframework.org/schema/context
https://www.springframework.org/schema/context/spring-context.xsd http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx.xsd">

    <!-- 引入其他的spring 配置文件 -->
    <import resource="common.xml"/>

</beans>
```

注意：在实际开发中，service 单独配置到一个文件中，dao 单独配置到一个文件中，然后在核心配置文件中引入，养成好习惯。

Spring 中的八大模式

简单工厂模式

BeanFactory的getBean()方法，通过唯一标识来获取 Bean 对象。是典型的简单工厂模式（静态工厂模式）；

工厂方法模式

FactoryBean是典型的工厂方法模式。在配置文件中通过factory-method属性来指定工厂方法，该方法是一个实例方法。

单例模式

Spring 用的是双重判断加锁的单例模式。请看下面代码，我们之前讲解 Bean 的循环依赖的时候见过：

@Nullable

```
protected Object getSingleton(String beanName, boolean allowEarlyReference) {  
    // Quick check for existing instance without full singleton lock  
    Object singletonObject = this.singletonObjects.get(beanName);  
    if (singletonObject == null && isSingletonCurrentlyInCreation(beanName)) {  
        singletonObject = this.earlySingletonObjects.get(beanName);  
        if (singletonObject == null && allowEarlyReference) {  
            synchronized (this.singletonObjects) {  
                // Consistent creation of early reference within full singleton lock  
                singletonObject = this.singletonObjects.get(beanName);  
                if (singletonObject == null) {  
                    singletonObject = this.earlySingletonObjects.get(beanName);  
                    if (singletonObject == null) {  
                        ObjectFactory<?> singletonFactory = this.singletonFactories.get(beanName);  
                        if (singletonFactory != null) {  
                            singletonObject = singletonFactory.getObject();  
                            this.earlySingletonObjects.put(beanName, singletonObject);  
                            this.singletonFactories.remove(beanName);  
                        }  
                    }  
                }  
            }  
        }  
    }  
    return singletonObject;  
}
```

老杜

代理模式

Spring 的 AOP 就是使用了动态代理实现的。

装饰器模式

JavaSE 中的 IO 流是非常典型的装饰器模式。

Spring 中配置 DataSource 的时候，这些 DataSource 可能是各种不同类型的，比如不同的数据库：Oracle、SQL Server、MySQL 等，也可能是不同的数据源：比如 apache 提供的 org.apache.commons.dbcp.BasicDataSource、spring 提供的 org.springframework.jndi.JndiObjectFactoryBean 等。

这时，能否在尽可能少修改原有类代码的情况下，做到动态切换不同的数据源？此时就可以用到装饰者模式。

Spring 根据每次请求的不同，将 dataSource 属性设置成不同的数据源，以到达切换数据源的目的。

Spring 中类名中带有 Decorator 和 Wrapper 单词的类，都是装饰器模式。

观察者模式

定义对象间的一对多的关系，当一个对象的状态发生改变时，所有依赖于它的对象都得到通知并自动更新。Spring 中观察者模式一般用在 listener 的实现。

Spring 中的事件编程模型就是观察者模式的实现。在 Spring 中定义了一个 **ApplicationListener** 接口，用来监听 **Application** 的事件，**Application** 其实就是 **ApplicationContext**，**ApplicationContext** 内置了几个事件，其中比较容易理解的是：**ContextRefreshedEvent**、**ContextStartedEvent**、**ContextStoppedEvent**、**ContextClosedEvent**。

策略模式

策略模式是行为性模式，调用不同的方法，适应行为的变化，强调父类的调用子类的特性。

`getHandler` 是 `HandlerMapping` 接口中的唯一方法，用于根据请求找到匹配的处理器。

比如我们自己写了 `AccountDao` 接口，然后这个接口下有不同的实现类：`AccountDaoForMySQL`，`AccountDaoForOracle`。对于 `service` 来说不需要关心底层具体的实现，只需要面向 `AccountDao` 接口调用，底层可以灵活切换实现，这就是策略模式。

模板方法模式

Spring 中的 `JdbcTemplate` 类就是一个模板类。它就是一个模板方法设计模式的体现。在模板类的模板方法 `execute` 中编写核心算法，具体的实现步骤在子类中完成。